

WYDZIAŁ INFORMATYKI I TELEKOMUNIKACJI
Instytut Informatyki

Praca dyplomowa magisterska

**ANALIZA RUCHU SIECIOWEGO APLIKACJI MOBILNYCH NA
PLATFORMIE ANDROID**

Mikołaj Kmiecik, nr albumu 147575

Promotor
dr inż. dr inż. Michał Kalewski

POZNAŃ 2025

Streszczenie

Celem pracy jest zbudowanie powtarzalnego środowiska do analizy ruchu aplikacji Android pod kątem bezpieczeństwa transmisji i prywatności. Pipeline łączy rejestrację na urządzeniu (PCAP/PCAPNG), klasyfikację protokołów (TLS, HTTP/1.1–3/QUIC) oraz korelację ze statyczną analizą (Exodus). Badania pokazują dominację HTTP/3/QUIC w aplikacjach społecznościowych, konserwatywne użycie TLS/HTTPS i niewiele domen w aplikacjach instytucjonalnych oraz typową telemetrię i zaplecze chmurowe w komercyjnych; odnotowano też ryzyko funkcjonalne (scenariusz Posnania). Ograniczenia: zmienność wersji i pinning/QUIC; rekomendacja: automatyczna kategoryzacja hostów i monitoring domen trzecich.

Słowa kluczowe: Android, analiza ruchu, TLS, HTTP/3/QUIC, prywatność, PCAP, telemetria.

Abstract

This thesis builds a reproducible framework for analyzing Android app traffic with a focus on transport security and privacy. The pipeline combines on-device capture (PCAP/PCAPNG), protocol classification (TLS, HTTP/1.1–3/QUIC), and correlation with static analysis (Exodus). Results show HTTP/3/QUIC dominance in social/media apps, conservative TLS/HTTPS with few third-party domains in institutional apps, and common telemetry plus cloud backends in commercial apps; a functional risk was observed (Posnania scenario). Limitations: version variability and pinning/QUIC; recommendation: automatic host categorization and continuous third-party monitoring.

Keywords: Android, network traffic analysis, TLS, QUIC, privacy, mitmproxy.

Spis treści

1	Wstęp	7
1.1	Motywacja	7
1.2	Cel i zakres pracy	9
2	Stan wiedzy i przegląd literatury	11
2.1	Analiza ruchu sieciowego	11
2.2	Inżynieria wsteczna i testy penetracyjne	11
2.3	Prywatność, uprawnienia i trackery	11
2.4	Zużycie zasobów i kontekst behawioralny	11
2.5	Podsumowanie	12
3	Podstawy teoretyczne	13
3.1	Architektura systemu Android	13
3.2	Model bezpieczeństwa	13
3.3	Protokoły	13
3.4	Techniki analizy ruchu	14
3.5	Prywatność i tracking	14
3.6	Narzędzia	14
4	Metodyka badań	17
4.1	Pytania badawcze i hipotezy	17
4.2	Zakres i dobór próby	17
4.3	Próba badawcza: lista aplikacji	17
4.4	Badane elementy (checklista/metryki)	18
4.4.1	Komunikacja i protokoły	18
4.4.2	Prywatność i strony trzecie	18
4.4.3	Zachowanie w tle i zużycie zasobów	18
4.4.4	Logi i artefakty	18
4.4.5	Powiązanie z narzędziami	19
4.5	Środowisko badawcze	19
4.5.1	Sprzęt i oprogramowanie	19
4.5.2	Bezpieczeństwo, etyka i zgodność	19
4.6	Procedura zbierania danych	19
4.6.1	Przechwytywanie przez proxy	19
4.6.2	Rejestracja pakietów	19
4.6.3	Postępowanie z pinningiem	19
4.7	Normalizacja i klasyfikacja ruchu	20
4.8	Metryki i kryteria oceny	20

4.8.1	Bezpieczeństwo transmisji	20
4.8.2	Prywatność i uprawnienia	20
4.8.3	Zużycie zasobów	20
4.9	Plan eksperymentów	20
4.10	Analiza danych	21
4.11	Ograniczenia	21
4.12	Replikowalność	21
5	Pierwsze spostrzeżenia przed badaniem właściwym	23
5.1	Inwentaryzacja aplikacji po resecie	23
5.2	Przechwytywanie radiowe Wiresharkiem w trybie monitorowania (niepowodzenie) .	24
5.3	MITM przez proxy	24
5.4	MITM z instrumentacją (Frida) wobec pinningu — próba nieudana	24
6	Eksperymenty: studia przypadków (PCAPdroid + Exodus)	27
6.1	Cel i założenia	27
6.2	Procedura (pojedyncza iteracja)	27
6.3	Instagram (wersja 394.0.0.46.81)	28
6.3.1	Wolumen i liczność wg protokołu	28
6.3.2	Największe cele (IP/port) wg wolumenu	28
6.3.3	Najczęstsze hosty (liczba przepływów)	29
6.3.4	Obserwacje jakościowe	29
6.3.5	Wyniki z Exodus Privacy [5]	29
6.3.6	Subiektywne spostrzeżenia	30
6.3.7	Podsumowanie	30
6.4	Facebook (wersja 526.1.0.66.75)	31
6.4.1	Wolumen i liczność wg protokołu	31
6.4.2	Największe cele (IP/port) wg wolumenu	31
6.4.3	Najczęstsze hosty (liczba przepływów)	32
6.4.4	Wyniki z Exodus Privacy (aplikacja)	32
6.4.5	Subiektywne spostrzeżenia	33
6.4.6	Podsumowanie	33
6.5	Posnania (wersja 1.1.1)	34
6.5.1	Wolumen i liczność wg protokołu	34
6.5.2	Największe cele (IP/port) wg wolumenu	34
6.5.3	Najczęstsze hosty (liczba przepływów)	34
6.5.4	Obserwacje jakościowe	35
6.5.5	Wyniki z Exodus Privacy [5]	35
6.5.6	Subiektywne spostrzeżenia	35
6.5.7	Podsumowanie	35
6.6	mObywatel (wersja 4.64.0 (4899))	36
6.6.1	Wolumen i liczność wg protokołu	36
6.6.2	Największe cele (IP/port) wg wolumenu	36
6.6.3	Najczęstsze hosty (liczba przepływów)	36
6.6.4	Obserwacje jakościowe	37
6.6.5	Wyniki z Exodus Privacy [5]	37
6.6.6	Subiektywne spostrzeżenia	37

6.6.7	Podsumowanie	38
6.7	Mobile USOS (wersja 2.0.3)	39
6.7.1	Wolumen i liczność wg protokołu	39
6.7.2	Największe cele (IP/port) wg wolumenu	39
6.7.3	Najczęstsze hosty (liczba przepływów)	40
6.7.4	Obserwacje jakościowe	40
6.7.5	Wyniki z Exodus Privacy [5]	40
6.7.6	Subiektywne spostrzeżenia	40
6.7.7	Podsumowanie	41
6.8	Spotify (wersja 9.0.72.967)	42
6.8.1	Wolumen i liczność wg protokołu	42
6.8.2	Największe cele (IP/port) wg wolumenu	42
6.8.3	Najczęstsze hosty (liczba przepływów)	43
6.8.4	Obserwacje jakościowe	43
6.8.5	Wyniki z Exodus Privacy [5]	43
6.8.6	Subiektywne spostrzeżenia	44
6.8.7	Podsumowanie	44
6.9	Voronoi (wersja 1.0.54)	45
6.9.1	Wolumen i liczność wg protokołu	45
6.9.2	Największe cele (IP/port) wg wolumenu	45
6.9.3	Najczęstsze hosty (liczba przepływów)	46
6.9.4	Obserwacje jakościowe	46
6.9.5	Wyniki z Exodus Privacy [5]	46
6.9.6	Subiektywne spostrzeżenia	47
6.9.7	Podsumowanie	47
6.10	TikTok (wersja 34.8.4)	48
6.10.1	Wolumen i liczność wg protokołu	48
6.10.2	Największe cele (IP/port) wg wolumenu	48
6.10.3	Najczęstsze hosty (liczba przepływów)	49
6.10.4	Obserwacje jakościowe	49
6.10.5	Wyniki z Exodus Privacy [5]	49
6.10.6	Subiektywne spostrzeżenia	49
6.10.7	Podsumowanie	50
7	Podsumowanie i wnioski końcowe	51
7.1	Synteza wyników	51
7.2	Wkład rozdziału „Pierwsze spostrzeżenia...”	52
7.3	Wnioski merytoryczne	53
7.4	Ograniczenia badania	53
7.5	Rekomendacje i kierunki dalszych prac	54
7.6	Konkluzja	54
	Bibliografia	55
	Spis tabel	57

Rozdział 1

Wstęp

1.1 Motywacja

Rynek aplikacji mobilnych stał się dominującym kanałem konsumpcji treści i usług cyfrowych. Dane o penetracji dostępu mobilnego (ok. 71% globalnej populacji posiada subskrypcję mobilną do 2023 r.) [4] oraz statystyki udziału ruchu WWW (ponad 50% globalnych *page views* pochodzi z urządzeń mobilnych) [7] pokazują, że to właśnie środowisko mobilne jest dziś kluczowym miejscem, w którym kształtują się wzorce komunikacji sieciowej i materializują ryzyka dla prywatności.

W ekosystemie Androida współlistnieje wiele bibliotek i zestawów SDK firm trzecich — od analityki i reklam, przez personalizację, po rozwiązania antyfraudowe. Każda z tych warstw wnosi własne punkty końcowe, protokoły i mechanizmy telemetrii, często działające poza świadomością użytkownika, a niekiedy również poza pełną kontrolą zespołów deweloperskich aplikacji. W efekcie rośnie złożoność oraz nieprzejrzystość przepływów danych.

Konsekwencją tego stanu rzeczy są konkretne wyzwania badawcze:

- a) uzyskanie przejrzystości, *jakie* protokoły, domeny i dostawcy są realnie wykorzystywani przez aplikacje,
- b) ocena ryzyka ekspozycji metadanych (np. hosty, czasy, rozmiary transferów) mimo powszechnego szyfrowania treści,
- c) porównywalność zachowań komunikacyjnych między kategoriami aplikacji (komunikatory, multimedia, finanse, gry),
- d) budowa powtarzalnych narzędzi i procedur, które nadążą za ewolucją protokołów (HTTP/2, HTTP/3/QUIC) oraz mechanizmami obronnymi (np. *certificate pinning*).

Wobec powyższego potrzebne jest uporządkowane, możliwie obiektywne podejście do *systematycznej* obserwacji i oceny ruchu sieciowego aplikacji: środowisko pomiarowe, które pozwala porównywać różne aplikacje i kategorie, zdefiniowany zestaw metryk (zarówno ilościowych, jak i jakościowych) oraz protokoły postępowania uwzględniające ograniczenia współczesnych technologii transportowych i zabezpieczeń. Tak rozumiana ramka badawcza stanowi bezpośrednią motywację dla niniejszej pracy.

Drugi wektor motywacyjny stanowi szybka adopcja nowych protokołów transportowych i warstw aplikacyjnych. QUIC (standaryzowany jako HTTP/3 [10]) przenosi część logiki transportu do przestrzeni użytkownika (UDP + TLS 1.3 [28]), co zmienia charakterystykę obserwowalnych cech przepływów: klasyczne estymatory opóźnień i retransmisji z poziomu TCP stają się mniej

bezpośrednie, a analiza pakietów jest utrudniona przez szyfrowanie większej liczby pól. W konsekwencji wcześniejsze proporcje protokołów publikowane w literaturze mogą być nieaktualne dla obecnych wersji popularnych aplikacji.

Kolejnym aspektem jest prywatność i otoczenie regulacyjne. RODO (GDPR), a także DMA/DSA w UE akcentują minimalizację i przejrzystość przetwarzania. Architektura „multi-endpoint” (API własne + analityka + reklamy + antyfraud + CDN/edge) prowadzi do gęstych, równoległych zestawów połączeń tuż po interakcji użytkownika. Nawet przy pełnym szyfrowaniu treści korelacja metadanych (częstotliwość, rozmiary odpowiedzi, domeny) może ujawniać wzorce użycia funkcji (pattern-of-life). Transparentność takich wzorców jest ograniczona zarówno dla użytkownika, jak i audytorów. Narzędzia systemowe dają uogólniony obraz, a techniki inwazyjne (hooking, dynamic instrumentation) bywają blokowane przez mechanizmy anty-tampering.

Istnieje także motywacja praktyczna stworzenia lekkiego, powtarzalnego środowiska badawczego balansującego między:

- **Zasięgiem obserwacji** (kompletność metadanych HTTP[S]),
- **Inwazyjnością** (brak konieczności rootowania, minimalne modyfikacje),
- **Zgodnością prawną i etyczną** (anonimizacja identyfikatorów, brak utrwalania treści wrażliwych).

Istniejące narzędzia (np. *mitmproxy*) zapewniają bazowy przechwyt, ale wymagają rozszerzeń o:

- **Normalizacja** – ujednolicenie formatu danych.
- **Klasyfikacja** – kategoryzacja ruchu sieciowego.
- **Agregacja** – zbieranie i analiza metryk.
- **Automatyzacja** – skrypty do interakcji z aplikacjami.
- **Anonimizacja** – ochrona danych osobowych.

Z naukowego punktu widzenia identyfikowane luki obejmują:

- L1** Brak bieżących, empirycznych pomiarów dystrybucji HTTP/1.1 vs HTTP/2 vs QUIC w popularnych aplikacjach.
- L2** Ograniczone porównania między kategoriami aplikacji a zachowaniami komunikacyjnymi.
- L3** Niedostatek jawnych metryk ryzyka prywatności bazujących wyłącznie na metadanych.
- L4** Fragmentaryczne dane o wpływie certificate pinning na pokrycie przechwyty.

Interesariusze:

- **Użytkownik** – zyskuje pośrednio poprzez privacy-by-design.
- **Deweloper** – możliwość audytu nadmiarowych zależności.
- **Audytor bezpieczeństwa** – metryki priorytetyzacji ryzyka.
- **Społeczność naukowa** – powtarzalny pipeline do dalszych badań.

1.2 Cel i zakres pracy

Cel: Celem pracy jest opracowanie i ewaluacja środowiska badawczego umożliwiającego systematyczną analizę bezpieczeństwa i prywatności aplikacji mobilnych na system Android. Praca koncentruje się na przechwytywaniu, klasyfikacji oraz ocenie ruchu sieciowego, a także na badaniu mechanizmów uwierzytelniania, uprawnień i zużycia zasobów.

Zakres pracy obejmuje:

- **Środowisko przechwytywania** – konfiguracja narzędzi takich jak *mitmproxy*, *Wireshark*, *tcpdump*, a także przygotowanie środowiska laboratoryjnego z wykorzystaniem emulatorów oraz zrootowanych urządzeń mobilnych.
- **Analiza ruchu sieciowego** – identyfikacja używanych protokołów (HTTP/1.1, HTTP/2, QUIC, TLS), badanie bezpieczeństwa transmisji danych (szyfrowanie, SSL pinning) oraz klasyfikacja przesyłanych informacji.
- **Reverse engineering i testy penetracyjne** – dekompilacja aplikacji z wykorzystaniem *APKTool*, *Jadx*, *Ghidra* oraz testy przy użyciu narzędzi takich jak *MobSF*, *Frida*, *Metasploit*.
- **Analiza uprawnień i prywatności** – badanie przydzielanych uprawnień, wykrywanie trackerów i komponentów reklamowych (*Exodus Privacy*, *App Inspector*) oraz ocena zarządzania danymi użytkowników.
- **Analiza zużycia zasobów** – pomiary obciążenia baterii i CPU aplikacji z użyciem *Battery Historian* oraz *Android Profiler*.
- **Eksperymenty** – testy na wybranych aplikacjach z różnych kategorii (m.in. komunikatory, aplikacje społecznościowe, bankowe, rządowe), analiza różnic w implementacji mechanizmów bezpieczeństwa.
- **Badanie wektorów ataków** – symulacja ataków typu *Man-in-the-Middle*, analiza przechwytywania danych lokalnych, scenariusze działania offline/online, testy odporności na automatyczne skrypty i boty.

Rozdział 2

Stan wiedzy i przegląd literatury

2.1 Analiza ruchu sieciowego

Badania pokazują, że ruch generowany przez aplikacje mobilne niesie znaczące wyzwania dla prywatności – nawet konfiguracje TLS nie chronią całkowicie, bo analizę można przeprowadzać na metadanych lub kanałach bocznych. W przeglądzie literatury poświęconym analizie ruchu mobilnego autorzy klasyfikują cele, miejsca przechwytywania oraz metody wykorzystania danych sieciowych do profilowania użytkowników [27]. Inne prace demonstrują, jak sieć może zostać wykorzystana do identyfikacji konkretnej aplikacji, nawet przy szyfrowaniu, wykorzystując uczenie maszynowe na podstawie cech pakietów [24]. Przykłady wykorzystania narzędzi takich jak *mitm-proxy* i *tcpdump* pokazują, jak można wychwycić i analizować zarówno HTTP jak i inne kanały sieciowe [13], [26].

2.2 Inżynieria wsteczna i testy penetracyjne

Inżynieria wsteczna aplikacji mobilnych, ułatwiająca badanie mechanizmów takich jak SSL pinning, jest szeroko stosowana. Wczesne prace wskazują na powszechne błędy konfiguracji SSL w aplikacjach Androidowych — ponad połowa analizowanych aplikacji korzystających z SSL miała błędne wdrożenia lub brak obsługi poprawnego pinningu [30]. W celu przeprowadzania dynamicznych testów bezpieczeństwa wykorzystuje się narzędzia takie jak *Frida*, *Drozer* czy *MobSF*, które wspierają zarówno analizę dynamiczną, jak i automatyzację [6], [15].

2.3 Prywatność, uprawnienia i trackery

Coraz więcej badań zwraca uwagę na skalę gromadzenia danych przez aplikacje mobilne. Systematyczne opracowania przeglądowe pokazują spektrum metod oceny prywatności (statycznych, dynamicznych i hybrydowych) oraz wskazują luki w zgodności z regulacjami [18]. Narzędzia takie jak *Exodus Privacy* umożliwiają analizę APK pod kątem wbudowanych trackerów i żądanych uprawnień [5]. Wyniki badań empirycznych pokazują, że aplikacje często żądają nadmiarowych uprawnień lub integrują zewnętrzne trackery reklamowe [2].

2.4 Zużycie zasobów i kontekst behawioralny

Nadmierne użycie zasobów (np. bateria, CPU) może wskazywać na obecność komponentów śledzących lub brak optymalizacji. Do pomiarów wykorzystuje się m.in. *Battery Historian* oraz

Android Profiler [20], [21]. Chociaż ten aspekt jest rzadziej eksplorowany w literaturze naukowej, podkreśla się jego znaczenie w identyfikacji ukrytych mechanizmów aplikacji [29].

2.5 Podsumowanie

Przegląd literatury wykazuje, że istnieje bogata baza wiedzy w obszarach analizy ruchu sieciowego, inżynierii wstecznej, testów penetracyjnych oraz badania prywatności aplikacji mobilnych. Jednocześnie wciąż brakuje holistycznego podejścia, które łączyłoby te wszystkie elementy — co stanowi niszę, którą niniejsza praca ma ambicję wypełnić.

Rozdział 3

Podstawy teoretyczne

3.1 Architektura systemu Android

Android jest otwartą, wielowarstwową platformą opartą na jądrze Linuksa. Stos oprogramowania obejmuje m.in. jądro (kernel) z mechanizmami bezpieczeństwa (np. SELinux), warstwę HAL (Hardware Abstraction Layer), biblioteki natywne, środowisko uruchomieniowe ART oraz framework Java/Kotlin z zestawem API, na którym budowane są aplikacje i usługi systemowe [3], [17]. W nowszych wersjach platformy pojawiły się dodatkowe izolatory, takie jak *SDK sandbox* (separowany proces dla wybranych SDK), co jeszcze bardziej ogranicza wpływ komponentów zewnętrznych na proces aplikacji [25].

3.2 Model bezpieczeństwa

Fundamentem bezpieczeństwa Androida jest *piaskownica* (application sandbox): każda aplikacja otrzymuje unikalny UID i działa w odseparowanym procesie z izolowanym przestrzennie systemem plików. Dodatkową ochronę zapewnia polityka MAC (SELinux), a także model uprawnień ograniczający dostęp do wrażliwych zasobów (lokalizacja, kamera, mikrofon) – w tym uprawnienia żądane w czasie działania (*runtime permissions*) [1], [16], [23]. Łańcuch zaufania domykają mechanizmy podpisywania aplikacji, Verified Boot/AVB oraz proces weryfikacji w ekosystemie sklepu (Play Protect, audyty polityk), co Google cyklicznie raportuje [8]. W kontekście komunikacji sieciowej część aplikacji implementuje *certificate pinning*, które utrudnia badaczowi pasywne przechwycenie treści HTTPS przez klasyczne proxy – poprawnie wdrożony pinning ma powodować odrzucenie połączenia z certyfikatem wystawionym przez zaufane, lecz nieprawidłowe (dla danej domeny) CA [12].

3.3 Protokoły

Przeglądane aplikacje korzystają z rodziny protokołów HTTP w różnych wersjach oraz z TLS do uwierzytelnienia i poufności.

- **HTTP/1.1** – klasyczny, tekstowy protokół nad TCP (przykładowe różnice semantyczne i transportowe względem nowszych wersji omawiają poniższe punkty HTTP/2/HTTP/3).
- **HTTP/2** – binarne ramkowanie, kompresja nagłówek i *multiplexing* wielu strumieni na jednym połączeniu TCP w celu redukcji opóźnień [9].
- **HTTP/3** – mapowanie semantyki HTTP na QUIC (UDP), eliminujące wpływ HoL na poziomie transportu i skracające zestawienie połączenia [10], [22].

- **TLS 1.3** – domyślny protokół kryptograficzny dla HTTPS, skracający *handshake*, upraszczający zestaw szyfrów i wspierający 0-RTT (z zastrzeżeniami bezpieczeństwa) [28].

3.4 Techniki analizy ruchu

W analizach bezpieczeństwa mobilnego wykorzystuje się zwykle dwa komplementarne podejścia: (1) *przechwytywanie i inspekcję ruchu* oraz (2) *instrumentację/dynamiczną obserwację aplikacji*. Do przechwytywania pakietów i strumieni wykorzystywane są m.in. *tcpdump* i *Wireshark*, a do pośredniczenia w ruchu HTTP(S) – *mitmproxy* (instalacja zaufanego CA na urządzeniu w celu deszyfracji ruchu) [11], [14], [26], [31]. W przypadku aplikacji z poprawnym *certificate pinning* wymagane bywa podejście z instrumentacją (np. *Frida*) lub środowisko z większą kontrolą (urządzenie z dostępem *root*/emulator, własny trust store), przy czym nowoczesne techniki (np. niestandardowi *trust managerzy*, natywne weryfikacje, integracja z *Play Integrity*) istotnie podnoszą poprzeczkę dla badań dynamicznych [6], [12]. Wybór między emulator a urządzeniem fizycznym wpływa na wierność zachowania (emulator ułatwia kontrolę i automatyzację; urządzenie fizyczne lepiej oddaje ograniczenia sprzętowe i integracje OEM) [11].

3.5 Prywatność i tracking

Problem prywatności w aplikacjach mobilnych obejmuje zarówno zakres i uzasadnienie uprawnień, jak i obecność komponentów stron trzecich (SDK analityczne/reklamowe). Do statycznego wykrywania trackerów i przeglądu uprawnień w APK wykorzystywane są m.in. *Exodus Privacy* oraz narzędzia deweloperskie Androida [5], [16]. Równolegle platforma rozwija rozwiązania mające ograniczyć śledzenie międzyaplikacyjne (np. *Privacy Sandbox on Android*) [19]. W praktyce ocena prywatności powinna łączyć analizę statyczną i dynamiczną (ruch sieciowy, zachowanie w tle), a także kontrolować zużycie zasobów (bateria/CPU) – symptomy nadmiernej telemetrii mogą ujawniać się jako wzrost aktywności sieciowej w tle czy anomalia energetyczne.

3.6 Narzędzia

W badaniach wykorzystano zestaw narzędzi do przechwytywania, inspekcji i korelacji ruchu sieciowego oraz do analizy statycznej aplikacji. Poniżej opisano ich rolę, wytwarzane artefakty i ograniczenia.

PCAPdroid (Android, tryb VPN)

- **Rola:** podstawowe narzędzie rejestracji ruchu *na urządzeniu* w badaniach właściwych; działa w oparciu o *VpnService* i przechwytuje pakiety bez rootowania.
- **Zastosowanie w pracy:** każda sesja aplikacji była rejestrowana do plików PCAPNG oraz agregowana do CSV (metryki: protokół, host, port, flows, wolumen RX/TX).
- **Artefakty:** pliki *.pcapng*, *.csv*; metryki per protokół/host/port wykorzystywane w tabelach i podsumowaniach.
- **Ograniczenia:** brak dostępu do treści szyfrowanych (TLS/QUIC); metryki oparte o meta-dane.

Wireshark (analiza i próby radiowe)

- **Rola:** inspekcja PCAP/PCAPNG oraz wstępne próby przechwytywania ramek w trybie *monitor*.
- **Zastosowanie w pracy:** manualna weryfikacja strumieni, klasyfikacja hostów i protokołów; podejście radiowe porzucono z uwagi na małą skuteczność w warunkach produkcyjnych.
- **Artefakty:** zrzuty ekranów/wnioski jakościowe; brak dodatkowych danych liczbowych poza tymi z PCAPdroida.
- **Ograniczenia:** trudności z dekrypcją WPA2/SAE (rotacja kluczy, brak materiału kryptograficznego); wysoki koszt operacyjny przy niskiej wartości poznawczej dla aplikacji mobilnych.

tcpdump (host/AP)

- **Rola:** uzupełniające przechwytywanie ruchu po stronie hosta lub punktu dostępowego (AP), szczególnie podczas konfiguracji środowiska i testów.
- **Zastosowanie w pracy:** weryfikacja, że ruch przechodzi przez kontrolowany węzeł; pomocnicze ślady do szybkiego podglądu (`tcpdump -i <iface> -w file.pcap`).
- **Artefakty:** .pcap do inspekcji w Wiresharku.
- **Ograniczenia:** jak wyżej — brak wglądu w treść TLS/QUIC; zależność od topologii sieci.

mitmproxy (HTTP(S) proxy, MITM)

- **Rola:** próby aktywnego *MITM* na HTTP/TLS-over-TCP (inspekcja nagłówków HTTP, mapowanie żądań).
- **Zastosowanie w pracy:** etap wstępny — klasyfikacja ruchu, kontrola DNS/trasowania przez własny węzeł.
- **Artefakty:** logi mitmproxy, ewentualne ślady żądań HTTP(S) dla aplikacji bez pinningu i bez HTTP/3.
- **Ograniczenia:** nieskuteczne wobec **HTTP/3/QUIC** oraz aplikacji z **pinningiem certyfikatów**; w badaniach właściwych kluczowe było przejście na rejestrację *on-device*.

Exodus Privacy (analiza statyczna APK)

- **Rola:** zewnętrzna analiza statyczna paczek aplikacji (sygnatury bibliotek *trackerów*, lista *uprawnień*).
- **Zastosowanie w pracy:** dla każdej aplikacji zestawiono liczbę trackerów i zakres uprawnień w badanej wersji; dane skorelowano z obserwacjami sieciowymi.
- **Artefakty:** wyniki (raporty WWW/PDF) cytowane w podrozdziałach; liczby w tabeli zbiorczej podsumowania.
- **Ograniczenia:** detekcja oparta o *sygnatury statyczne*; obecność biblioteki nie jest dowodem *aktywności* trackera w czasie sesji.

Frida (instrumentacja dynamiczna)

- **Rola:** próba obejścia niestandardowych weryfikacji TLS/pinningu w wybranych aplikacjach.
- **Zastosowanie w pracy:** etap wstępny — eksperymenty z hookami; próba nieudana ze względu na rozproszone ścieżki walidacji i ochrony anty-tamper.
- **Artefakty:** wnioski metodologiczne (ograniczona skuteczność bez głębokiej personalizacji skryptów).
- **Ograniczenia:** wysoka złożoność, podatność na mechanizmy anty-debug/anti-root; nie stosowano w badaniach właściwych.

Rozdział 4

Metodyka badań

4.1 Pytania badawcze i hipotezy

Celem metodyki jest rzetelne porównanie praktyk komunikacyjnych i mechanizmów ochrony prywatności w wybranych aplikacjach mobilnych na Androidzie. Odpowiadamy na pytania:

1. Jakie protokoły i wersje (HTTP/1.1, HTTP/2, HTTP/3/QUIC, TLS) wykorzystują badane aplikacje i czy połączenia są odpowiednio chronione [9], [10], [22], [28]?
2. Czy możliwy jest pasywny lub aktywny podsłuch ruchu z użyciem proxy/interceptora i jakie bariery występują (np. certificate pinning) [11], [12], [14]?
3. Jakie wzorce prywatności występują (trackery, punkty końcowe stron trzecich, zakres uprawnień) i jak korespondują z politykami platformy [5], [16], [19]?
4. Jaki jest profil zużycia zasobów (energia/CPU) w typowych scenariuszach użycia [20], [21]?

4.2 Zakres i dobór próby

Analizie podlegają popularne aplikacje z różnych kategorii (komunikacja, rozrywka, finanse, usługi publiczne). Dla każdej aplikacji realizujemy powtarzalne, zdefiniowane niżej scenariusze: uruchomienie, rejestracja/logowanie (jeśli dotyczy), przeglądanie treści, praca w tle (okno obserwacji), wylogowanie. Zakres nie obejmuje łamania zabezpieczeń serwerowych ani naruszeń regulaminów usług.

4.3 Próba badawcza: lista aplikacji

Na potrzeby badań przyjęto celowo dobraną próbę aplikacji mobilnych na Androidzie, obejmującą popularne kategorie (media społecznościowe, finanse, usługi publiczne, edukacja, handel/retail, rozrywka). Zestaw wyjściowy:

- **TikTok, Instagram, Facebook** — media społecznościowe;
- **Spotify** — streaming audio;
- **XTB** — finanse/inwestycje;
- **mObywatel** — usługi publiczne;
- **USOS** — system uczelniany/edukacja;

- **Posnania** — handel/galeria (aplikacja obiektu handlowego);
- **Voronoi, Gazet** — aplikacje lokalne/utility.

4.4 Badane elementy (checklista/metryki)

Ocena każdej aplikacji opiera się na zunifikowanej liście elementów, łączącej analizę ruchu, prywatności oraz zachowania w tle i zużycia zasobów:

4.4.1 Komunikacja i protokoły

- wersja protokołu aplikacyjnego i transportowego: HTTP/1.1, HTTP/2, HTTP/3/QUIC; identyfikacja po ALPN i charakterystyce strumieni;
- bezpieczeństwo transmisji: obecność TLS, wersja (w szczególności TLS 1.3), parametry sesji, błędy weryfikacji certyfikatu;
- *certificate pinning*: objawowa detekcja (odrzućcia łańcucha, brak wglądu w treść po zainstalowaniu CA użytkownika); próby obejścia tylko w granicach zgodnych z zaleceniami badawczymi (instrumentacja, emulator/środowisko z własnym trust store);
- profil ruchu: liczność żądań, kody odpowiedzi, rozmiary, timingi; rozróżnienie pierwszego planu vs. tło; intensyfikacja przepływów po interakcji i pory największej aktywności.

4.4.2 Prywatność i strony trzecie

- mapowanie domen/punktów końcowych i liczba unikalnych hostów/AS (proksy dla rozproszenia danych);
- identyfikacja trackerów/SDK reklamowych (analiza statyczna APK + weryfikacja dynamiczna w ruchu);
- korelacja uprawnień z obserwowanym zachowaniem (np. żądanie lokalizacji vs. transmisje geodanych); zgodność z dobrymi praktykami platformy;
- praktyki sesyjne: przechowywanie tokenów/ID, rotacja, wysyłka wrażliwych parametrów; ewentualne użycie plików cookie w komponentach webview.

4.4.3 Zachowanie w tle i zużycie zasobów

- aktywność sieciowa w tle (po zamknięciu/wylogowaniu, w oknie obserwacji);
- energia/CPU: metryki z narzędzi deweloperskich (timeline, wakelocki, sieć vs. czas) oraz anomalie energetyczne w korelacji ze zdarzeniami sieciowymi;
- scenariusze szczególne: tryb offline/online (spójność i bezpieczeństwo synchronizacji), zmiana łączności, restart aplikacji/urządzenia.

4.4.4 Logi i artefakty

- występowanie wrażliwych danych w logach/plikach tymczasowych; format i ochrona (szyfrowanie/dostęp);
- zgodność z zasadą minimalizacji (tylko niezbędne dane w logach oraz telemetrii).

4.4.5 Powiązanie z narzędziami

- przechwyt i inspekcja: mitmproxy (CA użytkownika), tcpdump/Wireshark (pcap);
- analiza statyczna/dynamiczna: Exodus Privacy, Frida/MobSF (w razie potrzeby);
- pomiary zasobów: Battery Historian, Android Profiler.

4.5 Środowisko badawcze

4.5.1 Sprzęt i oprogramowanie

Badania realizowane są w kontrolowanym labie z wydzieloną siecią bez dostępu osób trzecich. Jako urządzenia testowe wykorzystywane są telefony z Androidem (konfiguracje opisane w dzienniku badań) oraz host linuxowy pełniący rolę punktu dostępowego i kolektora ruchu. Intercepcję i rejestrację realizują: *mitmproxy* (CA użytkownika), *tcpdump/Wireshark*; infrastruktura sieciowa: *hostapd*, *dnsmasq* [14], [31].

Konfiguracja labu odzwierciedla praktyki rekomendowane przez OWASP MASTG dla przechwytywania ruchu HTTP(S) oraz alternatyw nie-HTTP [11]. W przypadkach aplikacji stosujących pinning rozważana jest instrumentacja dynamiczna (np. *Frida*) oraz uruchomienie w środowisku z większą kontrolą nad trust store (emulator, urządzenie z dostępem *root*) [6], [12].

4.5.2 Bezpieczeństwo, etyka i zgodność

Badania prowadzone są na własnych kontach testowych; nie gromadzimy wrażliwych danych osób trzecich. Zachowana jest minimalizacja danych oraz separacja środowisk. Wszelkie obserwacje interpretujemy przez pryzmat zasad ochrony danych (np. zgodność z wymogami RODO pozostaje poza zakresem eksperymentalnym, ale motywuje ocenę prywatności).

4.6 Procedura zbierania danych

4.6.1 Przechwytywanie przez proxy

Główną ścieżką jest ruch przez *mitmproxy* z zainstalowanym zaufanym CA użytkownika na urządzeniu. Dla każdej aplikacji rejestrujemy strumienie żądań i odpowiedzi, metadane (czas, metoda, kody, rozmiary), informacje o ALPN/SNI i właściwości połączenia TLS [11], [14]. Zwracamy uwagę na przypadki odrzucenia łańcucha certyfikatu (typowy objaw pinningu).

4.6.2 Rejestracja pakietów

Równolegle pozyskujemy zrzuty pakietów (*pcap*) z interfejsów sieciowych (*tcpdump/Wireshark*) dla analizy niskopoziomowej, klasyfikacji protokołów i weryfikacji QUIC/HTTP/3 na podstawie ALPN i charakterystyki strumieni [10], [22], [31].

4.6.3 Postępowanie z pinningiem

Jeżeli aplikacja implementuje certificate pinning poprawnie, atak MITM z CA użytkownika powinien się nie powieść [12]. W takich przypadkach dokumentujemy objawy (np. brak ruchu odszyfrowanego, błędy połączeń), a dalsze badanie może wymagać instrumentacji (hooking wybranych API) lub środowiska z modyfikowalnym trust store/emulacji [11], [12]. Zgodnie z MASTG,

obejście pinningu bywa możliwe, ale zależy od konkretnych implementacji i nie zawsze jest wykonalne bez głębszych modyfikacji aplikacji/OS [12].

4.7 Normalizacja i klasyfikacja ruchu

Po zebraniu danych normalizujemy je do poziomu *flow* (5-tuple + kierunek + ALPN) i *transakcji HTTP*. Ekstrahujemy następujące cechy:

- protokół transportowy i aplikacyjny (TCP/UDP; HTTP/1.1, HTTP/2, HTTP/3), wersja TLS, wynik weryfikacji certyfikatu [9], [10], [28];
- metryki ruchu (liczba żądań, kody odpowiedzi, rozmiary, czasy, intensyfikacje przepływów w tle/na pierwszym planie);
- informacje o domenach i dostawcach (SNI/Host), liczba unikalnych hostów/AS (proxymalna miara rozproszenia danych);
- sygnały prywatności (wywołania do znanych domen analitycznych/reklamowych, parametry identyfikatorów), triangulowane z wynikami analizy statycznej [5].

4.8 Metryki i kryteria oceny

4.8.1 Bezpieczeństwo transmisji

- udział ruchu szyfrowanego (TLS) i nieszyfrowanego (HTTP bez TLS);
- wersje TLS i negocjacje ALPN; wykrycie HTTP/3/QUIC [10], [22], [28];
- wskaźnik pinningu (objawowy): odsetek żądań, których nie udało się zdeszyfrować przy poprawnej konfiguracji CA użytkownika, z adnotacją o przyczynie [11], [12].

4.8.2 Prywatność i uprawnienia

- obecność trackerów i domen stron trzecich wskazana przez analizę statyczną APK (*Exodus Privacy*) oraz potwierdzona w ruchu dynamicznym [5];
- zakres i częstotliwość wywołań w tle do zewnętrznych punktów końcowych (okno obserwacji);
- mapowanie żądanych uprawnień na obserwowane zachowania sieciowe (np. dostęp do lokalizacji vs. transmisje geodanych) [16], [19].

4.8.3 Zużycie zasobów

- metryki z *Battery Historian*: aktywność komponentów, zdarzenia wakelock, sieć vs. czas, korelacje ze scenariuszami [20];
- metryki z *Android Profiler*: ślady CPU/pamięci/sieci w trakcie scenariuszy i pracy w tle [21].

4.9 Plan eksperymentów

Dla każdej aplikacji wykonujemy sekwencję:

1. **Przygotowanie**: czyste uruchomienie, weryfikacja uprawnień, instalacja CA użytkownika (jeśli dotyczy), reset liczników energii.

2. **Scenariusz interakcji:** rejestracja/logowanie, nawigacja po kluczowych widokach, generowanie typowego ruchu (pobieranie treści, wyszukiwanie), a następnie wyjście.
3. **Okno tła:** urządzenie pozostaje bez interakcji przez N minut w tej samej sieci; rejestrujemy ruch i zdarzenia energetyczne.
4. **Wylogowanie i zamknięcie:** kończymy sesję, sporządzamy checksumy artefaktów, archiwizujemy logi/profilery.

Każdy scenariusz powtarzamy co najmniej trzykrotnie i o różnych porach dnia, aby ograniczyć wpływ zmienności sieci i backendu.

4.10 Analiza danych

Zastosujemy statystyki opisowe (medianą i IQR dla zmiennych skośnych), wykresy czasowe i heatmapy przepływów. Dla wybranych porównań (np. porównanie intensywności wywołań w tle między kategoriami) stosujemy nieparametryczne testy różnic (np. Mann–Whitney/Kruskal–Wallis) z poprawką na wielokrotność, o ile rozkłady i liczebności na to pozwalają.

4.11 Ograniczenia

Ograniczenia obejmują: (i) zależność od wersji aplikacji i backendu w czasie pomiaru; (ii) niepełną obserwowalność w aplikacjach z rygorystycznym pinningiem/ochronami anty-tamper (*Play Integrity*, niestandardowi *trust managerzy*) [12]; (iii) wpływ środowiska (emulator vs. urządzenie fizyczne) na zachowanie [11]. Wyniki nie są uogólnieniem na całe kategorie, lecz charakterystyką zbadanych wersji.

4.12 Replikowalność

Zapewniamy spójne wersjonowanie artefaktów: konfiguracji *mitmproxy*, skryptów ekstrakcji cech, próbek *pcap* (zanonimizowanych) i raportów z *Battery Historian*. Tam, gdzie to możliwe, udostępniamy parametry komend i hashe binariów narzędzi (bez udostępniania samych APK). Dokumentujemy wersje protokołów i bibliotek (ALPN/TLS) [10], [22], [28].

TABELA 4.1: Mapa elementów badania do narzędzi i metryk

Element badania	Narzędzie / źródło	Przykładowe metryki / artefakty
Protokoły i wersje (HTTP/1.1, HTTP/2, HTTP/3/QUIC)	<i>tcpdump</i> , <i>Wireshark</i> [26], [31]	ALPN/SNI; rozróżnienie HTTP/1.1 vs HTTP/2 vs HTTP/3; cechy QUIC (DCID/SCID), liczba strumieni [9], [10], [22].
Bezpieczeństwo transmisji (TLS)	<i>Wireshark</i> , <i>mitmproxy</i> [14], [31]	wersja TLS (1.2/1.3), zestaw szyfrów, wynik walidacji certyfikatu, renegotiacje, 0-RTT (jeśli występuje) [28].
Intercepcja (proxy)	<i>mitmproxy</i> , MASTG [11], [14]	odsetek żądań odszyfrowanych; metody, kody HTTP, rozmiary, nagłówki (<i>Authorization</i> , <i>Set-Cookie</i>); TTFB i czas całkowity.

Element badania	Narzędzie / źródło	Przykładowe metryki / artefakty
Certificate pinning (objawowo)	<i>mitmproxy</i> + logi; opcj. <i>Frida</i> [6], [12]	odsetek połączeń odrzuconych mimo zaufanego CA użytkownika; wzorce błędów/wyjątków; klasy API do hookowania.
Ruch w tle / bez interakcji	<i>mitmproxy</i> , <i>tcpdump</i> [14], [26]	zadania/min w tle; rozkład czasowy (intensyfikacje przepływów), domeny docelowe, interwały retry/backoff.
Semantyka HTTP	<i>mitmproxy</i> [14]	metody (GET/POST/PUT...), kody odpowiedzi, rozmiary, kolejność/priorytetyzacja strumieni (H2/H3).
Trackery / strony trzecie	Exodus Privacy + korelacja w PCAP [5]	lista trackerów/SDK; mapowanie domen→dostawcy; identyfikatory/parametry telemetryczne w żądaniach.
Uprawnienia a zachowanie	manifest + Android Docs [16]	zestaw żądanych uprawnień; mapowanie uprawnienie→zachowanie (np. lokalizacja→transmisje geodanych); spójność deklaracji.
Zużycie energii	Battery Historian [20]	wakelocki, aktywność sieć czas, top procesy, zdarzenia radiowe; korelacje ze szczytami ruchu.
CPU / pamięć / sieć	Android Profiler [21]	%CPU, zdarzenia GC, użycie pamięci, throughput sieci; ślady w scenariuszach i w tle.
Scenariusze offline/online i degradacje	<i>mitmproxy</i> , <i>tcpdump</i> [14], [26]	retry, błędy synchronizacji, fallback protokołu/transportu, spójność danych po reconnect.
Logi i artefakty lokalne	<i>adb logcat</i> , inspekcja plików	obecność wrażliwych danych w logach/plikach tymczasowych; rotacja i ochrona (szyfrowanie/dostęp).

Rozdział 5

Pierwsze spostrzeżenia przed badaniem właściwym

5.1 Inwentaryzacja aplikacji po resecie

Cel: uzyskanie stanu wyjściowego urządzeń po przywróceniu ustawień fabrycznych oraz zebranie metadanych o aplikacjach zainstalowanych domyślnie, ze szczególnym uwzględnieniem możliwości kontroli wykorzystania danych sieciowych w tle.

Sprzęt: dwa telefony Samsung Galaxy S7 (po resecie do ustawień fabrycznych).

Po oczyszczeniu zrzutów z urządzeń uzyskano spójny zbiór **53** aplikacji domyślnych na każdym z telefonów.

Wyniki zbiorcze:

TABELA 5.1: Podsumowanie ustawień korzystania z danych sieciowych w tle (po resecie)

Kategoria	Liczba	Udział
Łączna liczba aplikacji	53	100%
Można zablokować dane w tle	44	83,0%
Nie można zablokować	9	17,0%
Domyślnie aktywny transfer	53	100%
Domyślnie zablokowany	0	0%

Legenda: „Można zablokować dane w tle” – aplikacje, dla których system Android udostępnia opcję ograniczenia transmisji danych w tle; „Nie można zablokować” – aplikacje i komponenty systemowe zachowujące dostęp do sieci w tle niezależnie od użytkownika; „Domyślnie aktywny transfer/zablokowany” – stan tuż po resecie fabrycznym; „Udział” – procent względem wszystkich zidentyfikowanych aplikacji ($N = 53$).

Aplikacje z nieograniczonym dostępem do danych w tle:

TABELA 5.2: Lista aplikacji, dla których nie można zablokować danych sieciowych w tle (po resecie)

Nazwa aplikacji
Konserwacja urządzenia
Samsung Cloud
Samsung Members
Sim AT
SmartThings
Udostępnianie łącza
Ustawienia
Workspace

Wniosek: stan „po resece” obejmuje wyłącznie aplikacje *domyślnie aktywne*, z których zdecydowaną większość (83%) można ręcznie ograniczyć pod kątem wykorzystania danych w tle. Około 17% stanowią jednak komponenty, które zachowują pełny dostęp do sieci w tle i nie podlegają tej kontroli.

Uwaga replikacyjna: powyższe wyniki pochodzą z pojedynczego zrzutu po resece na tym samym systemie operacyjnym w obu urządzeniach.

5.2 Przechwytywanie radiowe Wiresharkiem w trybie monitorowania (niepowodzenie)

Cel: pasywna obserwacja ruchu bez konfigurowania proxy na urządzeniu.

Sprzęt: laptop z Linuksem; karta Wi-Fi w trybie monitorowania; dwa Galaxy S7.

Metoda: nasłuch (monitor mode) podczas korzystania z Wi-Fi zabezpieczonego (WPA2), próba późniejszej dekrypcji ramek i analizy ruchu aplikacji.

Wynik: *niepowodzenie metody* w kontekście pełnej analizy treści: kluczowy ruch pozostaje szyfrowany (WPA2), brak materiału do bezstratnej dekrypcji (m.in. konieczność złapania handshake i posiadania parametrów sieci). Metoda okazała się niewspółmiernie pracochłonna i ograniczona względem celów projektu; zrezygnowano na rzecz podejścia proxy/pcap.

Wnioski: przy analizie aplikacji mobilnych efektywniejsze jest środowisko kontrolowane (AP + proxy/pcap) niż sniffing radiowy na produkcyjnej sieci.

5.3 MITM przez proxy

Cel: przechwytywanie i inspekcja ruchu HTTP(S) generowanego przez badane aplikacje.

Narzędzia: *mitmproxy*, *tcpdump*.

Sprzęt i sieć: laptop z systemem Debian skonfigurowany jako punkt dostępowy (hostapd + dnsmasq) oraz dwa telefony Samsung Galaxy S7 po resece do ustawień fabrycznych.

Metoda:

1. Konfiguracja AP i routingu; uruchomienie *mitmproxy* w trybie przechwytyującym oraz równoległe rejestrowanie pakietów za pomocą *tcpdump*.
2. Instalacja certyfikatu użytkownika *mitmproxy* na urządzeniach oraz próby uruchomienia i obsługi wybranych aplikacji.

Wynik: Przy podejściu pierwszym aplikacje odmawiały działania, wykrywając obecność proxy i blokując komunikację. Próba drugiego wariantu (instalacja certyfikatu i obejście pinningu) również zakończyła się niepowodzeniem: telefony odrzucały certyfikat podczas pracy, a dodatkowo podczas próby uzyskania uprawnień *root* w celu głębszej ingerencji doszło do uszkodzenia jednego z urządzeń. Sytuacja ta wymusiła zmianę metodologii i poszukiwanie alternatywnych rozwiązań.

Wnioski: samo zastosowanie proxy MITM daje bardzo ograniczoną obserwowalność wobec aplikacji stosujących pinning i/lub protokół QUIC. Niezbędne są inne podejścia, np. instrumentacja, przechwytywanie ruchu na poziomie systemowym (pcap) lub konwersja ruchu HTTP/3 na HTTP/2.

5.4 MITM z instrumentacją (Frida) wobec pinningu — próba nieudana

Cel: obejście mechanizmu *certificate pinning* i przywrócenie możliwości inspekcji treści żądań.

Narzędzia: *Frida* (*frida-server* oraz gotowe skrypty hookujące interfejsy API związane z walidacją certyfikatów).

Metoda: uruchomiono *frida-server* na urządzeniu i wstrzyknięto skrypty mające na celu przechwycenie wywołań metod weryfikujących certyfikaty oraz ich dynamiczne obejście.

Wynik: próba zakończyła się *niepowodzeniem*. Wstrzyknięte skrypty nie zadziałały w przypadku analizowanej aplikacji. Prawdopodobnymi przyczynami były: brak uprawnień *root*, aktywne mechanizmy SELinux oraz natywne implementacje weryfikacji certyfikatów (np. niestandardowe *TrustManager*), których nie obejmowały standardowe hooki.

Wnioski: w wielu nowoczesnych aplikacjach obejście pinningu bez dostępu do uprawnień *root* jest nierealne lub bardzo trudne. Alternatywami pozostają uruchamianie aplikacji w środowisku z pełnym dostępem administracyjnym (emulator, urządzenie z *root*) lub analiza ograniczona do metadanych transportowych (hosty, porty, czasy, wolumen danych), bez wglądu w same treści komunikatów.

Rozdział 6

Eksperymenty: studia przypadków (PCAPdroid + Exodus)

6.1 Cel i założenia

Celem eksperymentu jest empiryczna charakterystyka komunikacji sieciowej wybranych aplikacji oraz zestawienie obserwacji z danymi o uprawnieniach i trackerach (metodologia *Exodus Privacy*). Interesują nas:

1. rejestracja ruchu z użyciem **PCAPdroid** (tryb VPN, eksport CSV/PCAPNG),
2. agregacja metryk (protokół, host, port, wolumen),
3. korelacja z analizą statyczną **Exodus Privacy** (trackery i uprawnienia).

Uwaga: Exodus wykrywa sygnatury statyczne trackerów — nie jest to dowód ich aktywności w czasie działania.

6.2 Procedura (pojedyncza iteracja)

Zgodnie z metodyką, jedna iteracja (≈ 15 min) składa się z:

1. **Baseline OS (2 min)** – aplikacja nieuruchomiona - punkt odniesienia dla tła.
2. **Cold start (1 min)** – uruchomienie i ustabilizowanie ekranu głównego.
3. **Nawigacja (2 min)** – główne ekrany (feed, profil, wyszukiwarka).
4. **Media/operacja (1 min)** – krótkie odtwarzanie/przewijanie, wymuszenie większych transferów.
5. **Offline/online (4 min)** – 2 min offline + 2 min po reconnect; obserwacja retry/backoff.
6. **Tło (10 min)** – obserwacja po zamknięciu aplikacji.

6.3 Instagram (wersja 394.0.0.46.81)

Okno i wolumen

Okno eksperymentu: **2025-08-18 17:55:47–18:08:00** (≈ 12.2 min). Zarejestrowano **302** przepływy, wolumen $\approx$ **28.04 MiB** (RX $\approx$ **27.20 MiB**, TX $\approx$ **0.85 MiB**).

Przebieg badania: Scenariusz obejmował około **15 minut interakcji** z aplikacją, w tym: *nasłuch tła (2 min)*, logowanie, wylogowanie, oglądanie filmów i zdjęć, "serduszkowanie", nawigację po głównych ekranach (*offline i online*), zamknięcie aplikacji oraz nasłuch tła.

6.3.1 Wolumen i liczność wg protokołu

TABELA 6.1: Podsumowanie wg protokołu (Instagram, ta sesja)

Protokół	Flows	MiB (total)	B wysłane	B odebrane
QUIC	187	27.99	855,220	28,500,436
HTTPS	5	0.02	12,180	9,063
TCP	12	0.02	15,384	3,699
DNS	49	0.00	1,475	2,441
TLS	36	0.00	2,160	1,440
UDP	5	0.00	837	346
HTTP	8	0.00	480	320
Razem	302	28.04	888,736	28,517,745

Obserwacja: Ruch Instagrama jest silnie zdominowany przez QUIC/HTTP/3.

6.3.2 Największe cele (IP/port) wg wolumenu

TABELA 6.2: Największe kierunki ruchu (Instagram, ta sesja)

IP docelowe	Port	Proto	Flows	MiB
185.60.217.63	443	QUIC	137	27.52
157.240.251.63	443	QUIC	1	0.33
185.60.217.3	443	QUIC	35	0.08
185.60.217.20	443	QUIC	2	0.02
185.60.217.175	5222	TCP	11	0.02
157.240.251.11	443	QUIC	1	0.02
185.60.217.28	443	QUIC	1	0.01
157.240.0.63	443	QUIC	1	0.01
185.60.217.13	443	HTTPS	2	0.01
157.240.253.17	443	HTTPS	1	0.01

Obserwacja: Największy wolumen koncentruje się na adresie 185.60.217.63 (serwer Meta, Dublin). Występują też długotrwałe połączenia TCP/5222 (port MQTT) do 185.60.217.175, związane najprawdopodobniej z powiadomieniami/czatem.

6.3.3 Najczęstsze hosty (liczba przepływów)

TABELA 6.3: Top hosty (Instagram, liczba przepływów)

Host	Flows
graph.instagram.com	138
gateway.instagram.com	64
z-p42-chat-e2ee-ig.facebook.com	35
edge-mqtt.facebook.com	17
i.instagram.com	10
graph.facebook.com	5
i-fallback.instagram.com	5
scontent-ber1-1.cdninstagram.com	2
static.xx.fbcdn.net	2
lookaside.facebook.com	1

Obserwacja: Widoczna jest silna integracja z infrastrukturą Meta: poza domenami instagrama (graph, gateway, i.instagram.com) występują też hosty facebookowe (edge-mqtt, graph.facebook.com, lookaside).

6.3.4 Obserwacje jakościowe

- **Dominacja HTTP/3/QUIC.** Zdecydowana większość ruchu (liczebnie i wolumenowo) to QUIC na 443/UDP, co jest zgodne z trendem migracji warstwy mediów do HTTP/3 [10], [22].
- **Kanał utrzymania sesji.** Występuje równoległy kanał na **TCP/5222** (11–12 flows, mały wolumen), prawdopodobnie do komunikacji w czasie rzeczywistym/utrzymania połączenia.
- **Intensyfikacja przepływów po interakcji, następnie spadek aktywności.** Zliczenia per minuta (niezamieszczone tu dla zwięzłości) pokazują silny pik około 17:59 (setki nowych/kończących się flow), po którym następują krótkie sekwencje aktywności w tle ($\approx$ 18:05–18:06).
- **Niewielki ślad HTTP/1.1/HTTPS.** Jedynie kilka żądań; główna komunikacja i zasoby idą ścieżką QUIC.

6.3.5 Wyniki z Exodus Privacy [5]

Raport dla `com.instagram.android` (wersja 394.0.0.46.81, źródło Google Play) wykazał obecność **4 trackerów** oraz **69 uprawnień**.

Trackery.

- Facebook Flipper (analytics),
- Facebook Login (identification),
- Facebook Notifications,
- Google Analytics (analytics).

Uprawnienia — przykłady kluczowe:

- **Lokalizacja:** `ACCESS_FINE_LOCATION`, `ACCESS_MEDIA_LOCATION`,

- **Multimedia:** CAMERA, RECORD_AUDIO, READ/WRITE_EXTERNAL_STORAGE, READ_MEDIA_IMAGES/VIDEO,
- **Tożsamość/konta/telefonía:** GET_ACCOUNTS, READ_CONTACTS, READ_CALL_LOG, READ_PHONE_STATE, ANSWER_PHONE_CALLS,
- **Sieć i praca w tle:** ACCESS_NETWORK_STATE, WAKE_LOCK, RECEIVE_BOOT_COMPLETED, FOREGROUND_SERVICE_*.

Uwaga: pełna lista 69 uprawnień znajduje się w raporcie Exodusa (załącznik PDF/HTML).

6.3.6 Subiektywne spostrzeżenia

Obserwowalność treści. Podobnie jak w przypadku Facebooka, dominacja QUIC/HTTP/3 ($\approx 99,8\%$ wolumenu) mocno ogranicza możliwość wglądu w treść ruchu.

Zmienność wersji. Instagram, podobnie jak Facebook, jest często aktualizowany. Hosty, udział QUIC i zestaw uprawnień mogą różnić się między wersjami i regionami. Wyniki dotyczą wersji 394.0.0.46.81.

6.3.7 Podsumowanie

W badanej sesji Instagram wykazuje:

1. niemal całkowitą dominację QUIC/HTTP/3 w wolumenie ($\approx 99,8\%$),
2. intensywne użycie hostów `graph.instagram.com`, `gateway.instagram.com`, `chat-e2ee` (komunikacja, API, czat),
3. asymetrię RX/TX (27.2 vs 0.85 MiB) – użytkownik konsumował treści,
4. szerokie spektrum uprawnień (w tym lokalizacja, multimedia, telefonia) i obecność trackerów analitycznych.

6.4 Facebook (wersja 526.1.0.66.75)

Okno i wolumen

Okno eksperymentu: **2025-08-19 19:24:12–19:36:29** (≈ 12.28 min). Zarejestrowano **305** przepływów, wolumen $\approx$ **29.75 MiB** (**RX** $\approx$ **28.30 MiB**, **TX** $\approx$ **1.45 MiB**).

Przebieg badania: Scenariusz obejmował około **15 minut interakcji** z aplikacją, w tym: *nasłuch tła (2 min), logowanie, wylogowanie, oglądanie filmów i zdjęć, "lajkowanie", nawigację po głównych ekranach (offline i online), zamknięcie aplikacji oraz nasłuch tła*.

6.4.1 Wolumen i liczność wg protokołu

TABELA 6.4: Podsumowanie wg protokołu (Facebook, ta sesja)

Protokół	Flows	MiB (total)	B wysłane	B odebrane
QUIC	132	29.29	1,189,171	29,522,102
TLS	93	0.01	5,620	3,768
DNS	46	0.00	1,137	1,778
HTTPS	19	0.43	312,835	142,702
TCP	10	0.01	9,111	2,899
HTTP	5	0.00	300	200
Razem	305	29.75	1,518,174	29,673,449

Obserwacja: Ruch Facebooka jest silnie zdominowany przez QUIC/HTTP/3 (jak u Instagrama).

6.4.2 Największe cele (IP/port) wg wolumenu

TABELA 6.5: Największe kierunki ruchu (Facebook, ta sesja)

IP docelowe	Port	Proto	Flows	MiB
185.60.217.28	443	QUIC	50	25.46
157.240.210.14	443	QUIC	1	2.01
185.60.217.20	443	QUIC	79	1.79
185.60.217.3	443	HTTPS	2	0.20
157.240.210.16	443	HTTPS	8	0.15
157.240.210.3	443	HTTPS	3	0.03
185.60.217.13	443	HTTPS	2	0.02
185.60.217.28	443	HTTPS	1	0.02
157.240.0.13	443	QUIC	1	0.01
31.13.91.40	443	QUIC	1	0.01

Obserwacja: Największy wolumen koncentruje się na adresie 185.60.217.63 (serwer Meta, Dublin - jak u Instagrama).

6.4.3 Najczęstsze hosty (liczba przepływów)

TABELA 6.6: Top hosty (Facebook, liczba przepływów)

Host	Flows
graph.facebook.com	85
lookaside.facebook.com	54
chat-e2ee-mini.facebook.com	25
gateway.instagram.com	24
gateway.facebook.com	19
edge-mqtt.facebook.com	14
graph-fallback.facebook.com	14
gateway-fallback.facebook.com	9
edge-mqtt-fallback.facebook.com	2
*.fbcdn.net (różne)	5

Obserwacja: Analiza hostów pokazuje silną koncentrację ruchu na `graph.facebook.com` oraz powiązanych usługach (`lookaside`, `chat-e2ee-mini`, `gateway`). Wskazuje to na dominującą rolę centralnego API (`graph`) i serwisów pomocniczych (logowanie, czat, integracja z Instagramem). Dodatkowa obecność hostów typu `*-fallback` oraz `*.fbcdn.net` sugeruje stosowanie mechanizmów redundancji i serwerów CDN do dystrybucji treści multimedialnych.

6.4.4 Wyniki z Exodus Privacy (aplikacja)

Raport dla `com.facebook.katana` (wersja 525.0.0.53.51, Google Play) wskazuje obecność **8 trackerów** oraz **90 uprawnień**.

Trackery.

- Facebook Ads (advertisement),
- Facebook Audience (analytics),
- Facebook Flipper (analytics),
- Facebook Notifications,
- Facebook Places,
- Facebook Share,
- Google Analytics (analytics),
- Mapbox.

Uprawnienia — przykłady kluczowe:

- **Lokalizacja:** `ACCESS_COARSE_LOCATION`, `ACCESS_FINE_LOCATION`, `ACCESS_MEDIA_LOCATION`,
- **Multimedia:** `CAMERA`, `RECORD_AUDIO`, `READ/WRITE_EXTERNAL_STORAGE`, `READ_MEDIA_IMAGES/VIDEO`,
- **Tożsamość/konta/telefon:** `GET_ACCOUNTS`, `READ_CONTACTS`, `READ_PHONE_STATE`, `MANAGE_ACCOUNTS`,
- **Sieć i system:** `INTERNET`, `ACCESS_NETWORK_STATE`, `ACCESS_WIFI_STATE`, `NFC`, `WAKE_LOCK`, `SYSTEM_ALERT_WINDOW`,

- **Biometria i powiadomienia:** USE_BIOMETRIC, USE_FINGERPRINT, POST_NOTIFICATIONS.

Spostrzeżenia:

- Zidentyfikowano szerokie spektrum trackerów, głównie od samego Facebooka, a także Google i Mapbox.
- Zakres uprawnień jest bardzo szeroki (90), obejmując zarówno krytyczne elementy systemowe (kamera, mikrofon, kontakty, lokalizacja), jak i dodatkowe funkcje (NFC, kalendarz).
- Liczba i charakter uprawnień odpowiada aplikacji działającej jako główna platforma społecznościowa, ale równocześnie rodzi poważne obawy o prywatność użytkowników.

Uprawnienia — kategorie kluczowe. Dla Facebooka istotne z perspektywy prywatności są zwłaszcza:

- **Lokalizacja:** ACCESS_FINE_LOCATION, ACCESS_COARSE_LOCATION (precyzyjna/przybliżona lokalizacja).
- **Multimedia:** CAMERA, RECORD_AUDIO, dostęp do pamięci (READ/WRITE_EXTERNAL_STORAGE — jeśli występują w używanej wersji).
- **Tożsamość/konta/książka adresowa:** GET_ACCOUNTS, READ_CONTACTS.
- **Sieć/telefon i praca w tle:** ACCESS_NETWORK_STATE, WAKE_LOCK, RECEIVE_BOOT_COMPLETED.

Uwaga: dokładny zestaw i liczba uprawnień zależą od wersji aplikacji i kanału dystrybucji; w raporcie Exodusa dla `com.facebook.katana` należy je cytować dla konkretnej wersji aplikacji poddanej badaniu.

6.4.5 Subiektywne spostrzeżenia

Obserwowalność treści. Transport QUIC/UDP ogranicza klasyczne MITM/TLS-over-TCP; wgląd w semantykę HTTP jest bardzo ograniczony.

Zmienność wersji. Facebook to aplikacja o wysokiej dynamice wydawniczej; zestaw hostów, udział QUIC/H2 oraz uprawnienia mogą różnić się między wersjami i regionami. Liczby w tym rozdziale odnoszą się do konkretnej sesji i wersji aplikacji użytej w eksperymencie.

6.4.6 Podsumowanie

W badanej sesji Facebook wykazuje:

1. dominację QUIC/HTTP/3 w wolumenie ($\approx 98,5\%$),
2. charakterystyczne kanały usługowe (`graph/gateway/edge-mqtt`),
3. krótkotrwałe piki transferów związane z interakcjami użytkownika oraz ograniczoną aktywność w tle. Z perspektywy prywatności kluczowe są uprawnienia do lokalizacji, multimedii i pracy w tle oraz obecność komponentów analityczno-reklamowych.

6.5 Posnania (wersja 1.1.1)

Okno i wolumen

Okno eksperymentu: **2025-08-23 17:54:14–17:57:08** (≈ 2.9 min). Zarejestrowano **21** przepływy, wolumen $\approx$ **0.08 MiB** (RX $\approx$ **0.06 MiB**, TX $\approx$ **0.03 MiB**).

Przebieg badania: Scenariusz obejmował około **15 minut interakcji** z aplikacją, w tym: *nasłuch tła, start aplikacji, zmianę hasła, logowanie, dodanie rejestracji, nawigację po menu, wylogowanie, próbę błędnego logowania, ponowne logowanie, krótką pracę w trybie offline i online, ponowne wylogowanie oraz około 10 minut działania aplikacji w tle.*

6.5.1 Wolumen i liczność wg protokołu

TABELA 6.7: Podsumowanie wg protokołu (Posnania, ta sesja)

Protokół	Flows	MiB (total)	B wysłane	B odebrane
HTTPS	9	0.08	27,627	57,363
DNS	12	0.00	260	468
Razem	21	0.08	27,887	57,831

Obserwacja: Ruch aplikacji jest w całości oparty o **HTTPS** (brak QUIC oraz HTTP w badanym oknie czasowym).

6.5.2 Największe cele (IP/port) wg wolumenu

TABELA 6.8: Największe kierunki ruchu (Posnania, ta sesja)

IP docelowe	Port	Proto	Flows	MiB
104.21.42.187	443	HTTPS	1	0.04
34.120.195.249	443	HTTPS	5	0.02
52.26.132.221	443	HTTPS	2	0.01
172.67.208.92	443	HTTPS	1	0.01
10.215.173.2	53	DNS	12	0.00

Obserwacja: Cele to wyłącznie port **443/TCP** (HTTPS) oraz standardowe **DNS/53**. Brak nietypowych portów.

6.5.3 Najczęstsze hosty (liczba przepływów)

TABELA 6.9: Top hosty (Posnania, liczba przepływów)

Host	Flows
api.yourb.app	10
o4505784913690624.ingest.sentry.io	6
m.stripe.com	3
updates.yourb.app	2

Obserwacja: Widoczne są domeny **yourb.app** (API/aktualizacje), **sentry.io** (raportowanie błędów) oraz **stripe.com** (płatności).

6.5.4 Obserwacje jakościowe

- **Brak QUIC/HTTP/3.** Cała komunikacja po 443/TCP (TLS).
- **Brak HTTP.** W badanej sesji nie obserwowano niezaszyfrowanych żądań HTTP.
- **Integracje zewnętrzne.** Ruch do `sentry.io` (crash reporting) i `stripe.com` (płatności) jest zgodny z funkcjonalnością aplikacji.

6.5.5 Wyniki z Exodus Privacy [5]

Raport dla `com.yourbuildingapp.app.pośnania` (wersja 1.1.1, Google Play) wskazuje obecność **1 trackera** (*Sentry* — crash reporting) oraz **18 uprawnień**.

Uprawnienia — przykłady kluczowe:

- **Kamera i multimedia:** CAMERA, READ_EXTERNAL_STORAGE, WRITE_EXTERNAL_STORAGE, READ_MEDIA_IMAGES, READ_MEDIA_VIDEO.
- **Tożsamość/telefon:** READ_PHONE_STATE.
- **Interfejs/system:** SYSTEM_ALERT_WINDOW, RECEIVE_BOOT_COMPLETED, SCHEDULE_EXACT_ALARM, POST_NOTIFICATIONS.
- **Sieć:** INTERNET, ACCESS_NETWORK_STATE, ACCESS_WIFI_STATE.
- **Biometria:** USE_BIOMETRIC, USE_FINGERPRINT.

Spostrzeżenia: integracja z Sentry jest zgodna z profilem aplikacji, jednak szeroki zestaw uprawnień (kamera, biometria, alarmy systemowe) może być postrzegany jako nadmiarowy względem prostej funkcji aplikacji centrum handlowego.

6.5.6 Subiektywne spostrzeżenia

Obserwowalność treści. Ze względu na TLS treści żądań/odpowiedzi nie są dostępne; brak oznak jawnego przesyłania danych.

Hipotezy dot. bezpieczeństwa (hipoteza wymagająca osobnych badań w celu klaryfikacji). Podczas korzystania z aplikacji spostrzeżono możliwość wycieku informacji danych wrażliwych. Dodawanie tablic rejestracyjnych pojazdów nie wymaga wykazania własności samochodu. Dodać można każdą rejestrację, a następnie śledzić zarejestrowane na nią wjazdy i wyjazdy z parkingu. Na podstawie samego ruchu sieciowego *nie można* potwierdzić „otwartej”/słabo szyfrowanej bazy danych — wymaga to analizy on-device (np. `SQLite/Room`, manifest, reverse engineering APK) oraz testów dynamicznych (np. `Frida`, `mitmproxy`, sprawdzenie pinu certyfikatu).

6.5.7 Podsumowanie

W badanej sesji Pośnania wykazuje:

1. ruch wyłącznie po HTTPS (bez QUIC), niewielki wolumen (≈ 0.08 MiB) i zapytania DNS,
2. integrację z `sentry.io` (telemetria błędów) i `stripe.com` (płatności),
3. brak śladów HTTP w badanym oknie czasowym; dalsze badania wymagane, by ocenić bezpieczeństwo lokalnej bazy danych.

6.6 mObywatel (wersja 4.64.0 (4899))

Okno i wolumen

Okno eksperymentu: **2025-08-24 13:23:03–13:26:00** (≈ 2.9 min). Zarejestrowano **13** przepływy, wolumen $\approx$ **0.21 MiB** (RX $\approx$ **0.15 MiB**, TX $\approx$ **0.06 MiB**).

Przebieg badania: Scenariusz obejmował około **15 minut interakcji** z aplikacją, w tym: *nasłuch tła (2 min)*, *logowanie (biometria + hasło)*, *wyświetlenie numeru PESEL*, *wgląd w dane wrażliwe (dowód, prawo jazdy, legitymacja studencka, dane pojazdu)*, *nawigacja po głównych ekranach (offline i online)*, *zamknięcie aplikacji oraz nasłuch tła (ok. 10 min)*.

6.6.1 Wolumen i liczność wg protokołu

TABELA 6.10: Podsumowanie wg protokołu (mObywatel, ta sesja)

Protokół	Flows	MiB (total)	B wysłane	B odebrane
HTTPS	3	0.18	51,439	134,763
TLS	4	0.02	13,802	10,017
HTTP	1	0.01	630	8,157
DNS	5	0.00	327	407
Razem	13	0.21	66,198	153,344

Obserwacja: Dominuje **HTTPS/TLS**; pojedynczy ślad **HTTP/80** dotyczy hosta Google (prawdopodobnie kontrola łączności/zasób systemowy) i nie wskazuje na przesyłanie treści aplikacyjnych w badanym oknie czasowym.

6.6.2 Największe cele (IP/port) wg wolumenu

TABELA 6.11: Największe kierunki ruchu (mObywatel, ta sesja)

IP docelowe	Port	Proto	Flows	MiB
185.41.93.206	443	HTTPS	3	0.18
185.32.48.239	3000	TLS	4	0.02
142.250.203.195	80	HTTP	1	0.01
10.215.173.2	53	DNS	5	0.00

Obserwacja: Ruch koncentruje się na 443/TCP do usług państwowych; widoczny jest także kanał TLS na nietypowym porcie **3000** (telemetry/błędy).

6.6.3 Najczęstsze hosty (liczba przepływów)

TABELA 6.12: Top hosty (mObywatel, liczba przepływów)

Host	Flows
api.mobywatel.gov.pl	6
sentry.puch.coi.gov.pl	5
www.gstatic.com	2

Obserwacja: `api.mobywatel.gov.pl` dominuje; `sentry.puch.coi.gov.pl` wskazuje na wewnętrzny system raportowania błędów; `www.gstatic.com` występuje pomocniczo (zasoby/kontrola łączności).

6.6.4 Obserwacje jakościowe

- **Szyfrowanie transportowe:** właściwa komunikacja aplikacji po TLS/443; brak treści w badanym oknie czasowym.
- **Brak QUIC/HTTP/3** w zarejestrowanym wycinku — kanał klasyczny TLS/TCP.
- **Telemetria:** obecność endpointu `sentry.puch.coi.gov.pl` (raportowanie błędów); wolumen niewielki.
- **HTTP/80 do Google:** pojedynczy flow (Google), typowy dla sprawdzeń łączności; brak wskazań na transfer danych aplikacji w badanym oknie czasowym.

6.6.5 Wyniki z Exodus Privacy [5]

Raport dla `pl.nask.mobywatel` (wersja 4.64.0 (4899), źródło Google Play) wskazuje **1 tracker** (*Sentry* — crash reporting) oraz **20 uprawnień**.

Uprawnienia — przykłady kluczowe:

- **Lokalizacja:** `ACCESS_COARSE_LOCATION`, `ACCESS_FINE_LOCATION`,
- **Multimedia/kamera/pamięć:** `CAMERA`, `READ_EXTERNAL_STORAGE`, `WRITE_EXTERNAL_STORAGE`,
- **Biometria:** `USE_BIOMETRIC`, `USE_FINGERPRINT`,
- **Telefonia/tożsamość:** `READ_PHONE_STATE`,
- **Sieć i usługi:** `INTERNET`, `ACCESS_NETWORK_STATE`, `ACCESS_WIFI_STATE`, `FOREGROUND_SERVICE`, `RECEIVE_BOOT_COMPLETED`, `WAKE_LOCK`, `NFC`, `POST_NOTIFICATIONS`, `GET_TASKS`,
- **Powiadomienia/FCM:** `RECEIVE`, `pl.nask.mobywatel.DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION`.

Spostrzeżenia: brak większej ilości trackerów potwierdza szczególną wagę, jaką aplikacja przykłada do prywatności. Zestaw uprawnień jest szeroki, ale adekwatny do funkcji (obsługa dokumentów tożsamości, NFC, biometria).

6.6.6 Subiektywne spostrzeżenia

Obserwowalność treści. Ze względu na TLS treści zapytań/odpowiedzi nie są dostępne; nie widać przesyłania danych wrażliwych w badanym oknie czasowym.

Praktyki bezpieczeństwa. W analizowanym wycinku widać konsekwentne użycie HTTP-S/TLS do usług `*.gov.pl`, brak nietypowych punktów końcowych poza kanałem telemetrycznym (TLS/3000) i brak szerokich odwołań do domen trzecich. To zachowanie spójne z wysokim standardem bezpieczeństwa na warstwie transportowej. Pełna ocena (np. pinning certyfikatów, mechanizmy anti-tamper/anti-root, magazyn kluczy, ochrona ekranu) wymaga analizy APK oraz testów dynamicznych.

6.6.7 Podsumowanie

W badanej sesji mObywatel wykazuje:

1. dominację TLS/HTTPS; wolumen (≈ 0.21 MiB), **13** przepływów w ~ 2.9 min,
2. koncentrację ruchu na `api.mobywatel.gov.pl` oraz wewnętrzną telemetrię `sentry.puch.coi.gov.pl`,
3. pojedynczy flow HTTP/80 do zasobów Google (nie dotyczy danych wrażliwych aplikacji).

6.7 Mobile USOS (wersja 2.0.3)

Okno i wolumen

Okno eksperymentu: **2025-08-24 13:43:09–13:47:29** (≈ 4.3 min). Zarejestrowano **200** przepływów, wolumen $\approx$ **2.88 MiB** (RX $\approx$ **2.47 MiB**, TX $\approx$ **0.40 MiB**).

Przebieg badania: Scenariusz obejmował około **15 minut interakcji** z aplikacją, w tym: *nasłuch tła (2 min)*, *logowanie*, *wyświetlenie numeru PESEL*, *wgląd w dane wrażliwe (PESEL, numer albumu)*, *wysłanie maila*, *nawigację po głównych ekranach (offline i online)*, *zamknięcie aplikacji (wylogowanie)* oraz *nasłuch tła (ok. 10 min)*.

6.7.1 Wolumen i liczność wg protokołu

TABELA 6.13: Podsumowanie wg protokołu (USOS, ta sesja)

Protokół	Flows	MiB (total)	B wysłane	B odebrane
HTTPS	173	2.87	421,868	2,592,198
TLS	22	0.00	1,320	880
DNS	5	0.00	360	725
Razem	200	2.88	423,548	2,593,803

Obserwacja: Ruch USOS w tej sesji jest zdominowany przez **HTTPS/TLS** (brak QUIC i brak HTTP w badanym oknie czasowym).

6.7.2 Największe cele (IP/port) wg wolumenu

TABELA 6.14: Największe kierunki ruchu (USOS, ta sesja)

IP docelowe	Port	Proto	Flows	MiB
150.254.5.110	443	HTTPS	25	0.91
216.58.208.206	443	HTTPS	5	0.90
142.250.186.196	443	HTTPS	14	0.54
150.254.5.52	443	HTTPS	126	0.51
142.250.186.42	443	HTTPS	2	0.01
142.250.203.131	443	HTTPS	1	0.01
150.254.5.52	3000	TLS	20	0.00
150.254.5.110	3000	TLS	2	0.00

Obserwacja: Największy wolumen kierowany jest do hostów na porcie **443/TCP** (HTTPS); widoczny jest też kanał **TLS/3000** o znikomym wolumenie (telemetria/błędy).

6.7.3 Najczęstsze hosty (liczba przepływów)

TABELA 6.15: Top hosty (USOS, liczba przepływów)

Host	Flows
usosapps.put.poznan.pl	147
ppulse.put.poznan.pl	28
www.google.com	14
clients4.google.com	6
firebaseinstallations.googleapis.com	3
firebase-settings.crashlytics.com	2

Obserwacja: Dominują hosty uczelniane (usosapps.put.poznan.pl, ppulse.put.poznan.pl); obecne są też serwisy Google (instalacje Firebase, Crashlytics) oraz www.google.com (zasoby/kontrola łączności).

6.7.4 Obserwacje jakościowe

- **Brak QUIC/HTTP/3.** Komunikacja aplikacyjna po TLS/443; treść niedostępna.
- **Brak HTTP.** Nie odnotowano żądań nieszyfrowanych.
- **Integracje zewnętrzne.** Połączenia do usług Google (Firebase/Crashlytics) o małym wolumenie są zgodne z diagnostyką/aplikacyjną telemetrią.

6.7.5 Wyniki z Exodus Privacy [5]

Raport dla pl.edu.usos.mobilny (wersja 2.0.3, Google Play) wskazuje obecność **1 trackera** — *Google CrashLytics* (crash reporting) oraz **13 uprawnień**.

Uprawnienia — przykłady kluczowe:

- **Kamera/kalendarium:** CAMERA, READ_CALENDAR, WRITE_CALENDAR,
- **Biometria:** USE_BIOMETRIC, USE_FINGERPRINT,
- **Sieć/system:** INTERNET, ACCESS_NETWORK_STATE, NFC, POST_NOTIFICATIONS, WAKE_LOCK,
- **Inne:** CHECK_LICENSE, RECEIVE (FCM), pl.edu.usos.mobilny.DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION.

Spostrzeżenia: Jedyne tracker ma charakter czysto diagnostyczny; zestaw uprawnień obejmuje zarówno funkcje systemowe (NFC, notyfikacje), jak i elementy fakultatywne (kalendarz).

6.7.6 Subiektywne spostrzeżenia

Obserwowalność treści. Warstwa transportowa jest poprawnie szyfrowana (TLS); brak widoczności treści żądań/odpowiedzi oraz brak jawnego HTTP.

Praktyki bezpieczeństwa. W analizowanej sesji widać konsekwentne użycie **HTTPS/TLS** do hostów uczelnianych oraz ograniczoną telemetrię (Crashlytics/Firebase). Dalsza ocena (pinning certyfikatów, mechanizmy anti-root/anti-tamper, magazyn kluczy) wymaga analizy APK i testów dynamicznych.

6.7.7 Podsumowanie

W badanej sesji USOS wykazuje:

1. dominację TLS/HTTPS; wolumen (≈ 2.88 MiB), **200** przepływów w ~ 4.3 min,
2. koncentrację ruchu na hostach uczelnianych (`usosapps.put.poznan.pl`, `ppulse.put.poznan.pl`) oraz niewielką telemetrię do usług Google,
3. brak śladów HTTP w badanym oknie czasowym oraz brak QUIC/HTTP/3 w zarejestrowanym oknie.

6.8 Spotify (wersja 9.0.72.967)

Okno i wolumen

Okno eksperymentu: **2025-08-24 14:09:43–2025-08-24 14:11:57** (≈ 2.2 min). Zarejestrowano **72** przepływy, wolumen $\approx$ **12.42 MiB** (RX $\approx$ **11.85 MiB**, TX $\approx$ **0.58 MiB**).

Przebieg badania: Scenariusz obejmował: *logowanie, odtwarzanie kilku różnych utworów, przewijanie/seek, nawigację po ekranach głównych, wyszukiwanie, przeglądanie i edycję playlist (dodawanie utworów), wylogowanie.*

6.8.1 Wolumen i liczność wg protokołu

TABELA 6.16: Podsumowanie wg protokołu (Spotify, ta sesja)

Protokół	Flows	MiB (total)	B wysłane	B odebrane
HTTPS	42	12.36	589,260	12,367,835
TCP	5	0.06	10,391	49,839
DNS	23	0.01	1,563	4,299
MDNS	1	0.00	730	0
SSDP	1	0.00	1,530	0
Razem	72	12.42	603,474	12,421,973

Obserwacja: Ruch Spotify jest zdominowany przez **HTTPS** (strumienie audio i zasoby CDN). W tym wycinku nie zaobserwowano QUIC; widoczny jest niewielki ślad **TCP** (kontrola/utrzymanie) oraz **DNS**, **mDNS** i **SSDP** (integracje urządzeń domowych).

6.8.2 Największe cele (IP/port) wg wolumenu

TABELA 6.17: Największe kierunki ruchu (Spotify, ta sesja)

IP docelowe	Port	Proto	Flows	MiB
213.158.203.250	443	HTTPS	1	5.93
146.75.118.248	443	HTTPS	1	2.30
146.75.118.250	443	HTTPS	2	1.87
35.186.224.28	443	HTTPS	2	1.48
35.186.224.24	443	HTTPS	17	0.42
213.158.203.240	443	HTTPS	8	0.18
213.158.203.200	443	HTTPS	1	0.11
35.186.224.44	443	HTTPS	5	0.04
34.158.1.133	4070	TCP	1	0.02
34.158.255.62	443	TCP	1	0.01

Obserwacja: Największy wolumen trafia do adresów ***.spotify.com/*.spotifycdn.com** na **443/TCP** (CDN i API).

6.8.3 Najczęstsze hosty (liczba przepływów)

TABELA 6.18: Top hosty (Spotify, liczba przepływów)

Host	Flows
spclient.wg.spotify.com	17
image-cdn-ak.spotifycdn.com	9
gew4-dealer.spotify.com	6
ap-gew4.spotify.com	4
image-cdn-fa.spotifycdn.com	3
gew4-spclient.spotify.com	3
pickasso.spotifycdn.com	3
ap-gue1.spotify.com	2
ap-guc3.spotify.com	2
firebaseanalytics-pa.googleapis.com	2

Obserwacja: Widoczne są typowe domeny Spotify: `spclient.wg.spotify.com` (API/sterowanie), `spotifycdn.com` (okładki/zasoby), regionalne `gew*/ap-*` (dealery API). Okazjonalne żądania do `firebaseanalytics-pa.googleapis.com` wskazują na niewielką telemetrię.

6.8.4 Obserwacje jakościowe

- **Strumień audio po HTTPS** — w tym rzucie brak QUIC/HTTP/3; transport realizowany klasycznym TLS/TCP.
- **Integracje urządzeń** — obecność **mDNS/SSDP** sugeruje wykrywanie urządzeń w sieci lokalnej (Cast/AirPlay/Connect).
- **Asymetria RX/TX** — przewaga danych odbieranych (11.85 MiB) nad wysyłanymi (0.58 MiB), typowa dla odtwarzania multimediiów.

6.8.5 Wyniki z Exodus Privacy [5]

Raport dla `com.spotify.music` (wersja 9.0.72.967, Google Play) ujawnił **4 trackery** oraz **48 uprawnień**.

Trackery:

- **Branch** — atrybucja i linki dynamiczne,
- **ComScore** — pomiar oglądalności i statystyk,
- **Google CrashLytics** — diagnostyka i stabilność aplikacji,
- **Google Firebase Analytics** — analityka użycia.

Uprawnienia (wybrane):

- **Multimedia:** `RECORD_AUDIO`, `READ_MEDIA_AUDIO`, `WRITE_EXTERNAL_STORAGE`,
- **Łączność:** `BLUETOOTH_*`, `ACCESS_WIFI_STATE`, `CHANGE_NETWORK_STATE`,
- **Reklama i identyfikacja:** `AD_ID`, `ACCESS_AD SERVICES_AD_ID`.

Spostrzeżenia: obecność czterech trackerów odpowiada modelowi biznesowemu aplikacji muzycznej opartej na reklamie i analityce. Zakres uprawnień jest bardzo szeroki, obejmujący zarówno multimedia, jak i identyfikatory reklamowe.

6.8.6 Subiektywne spostrzeżenia

Obserwowalność treści. Zaszyfrowany strumień HTTPS uniemożliwia wgląd w treść audio lub zapytania API; analiza ogranicza się do metadanych (hosty, porty, wolumen).

Uwagi dot. bezpieczeństwa. Zachowanie sieciowe (TLS/443, brak jawnego HTTP, ograniczona telemetria) jest spójne z dojrzałą aplikacją medialną. Ocena pinningu certyfikatów, DRM czy mechanizmów anti-tamper wymaga analizy APK/testów dynamicznych.

6.8.7 Podsumowanie

W badanej sesji Spotify wykazuje:

1. dominację HTTPS (≈ 12.42 MiB w **72** flow, okno 2.2 min),
2. koncentrację na hostach `*.spotify.com/*.spotifycdn.com` oraz aktywność mDNS/SSDP (urządzenia w sieci lokalnej),
3. zestaw trackerów i uprawnień zgodny z raportem Exodusa (4/48).

6.9 Voronoi (wersja 1.0.54)

Okno i wolumen

Okno eksperymentu: **2025-08-24 14:31:51–2025-08-24 14:37:25** (≈ 5.6 min). Zarejestrowano **465** przepływy, wolumen $\approx$ **9.61 MiB** (RX $\approx$ **9.21 MiB**, TX $\approx$ **0.39 MiB**).

Przebieg badania: Scenariusz obejmował: *nasłuch tła (2 min)*, *utworzenie konta*, *logowanie*, *nawigację po ekranach (online/offline)*, *wyszukiwanie*, *wylogowanie*.

6.9.1 Wolumen i liczność wg protokołu

TABELA 6.19: Podsumowanie wg protokołu (Voronoi, ta sesja)

Protokół	Flows	MiB (total)	B wysłane	B odebrane
HTTPS	174	9.53	373,137	9,622,074
TLS	226	0.06	33,132	33,984
DNS	42	0.01	1,881	3,620
TCP	23	0.00	2,420	2,144
Razem	465	9.61	410,570	9,661,822

Obserwacja: Ruch aplikacji jest zdominowany przez **HTTPS**; kanały **TLS/TCP** mają znikomy wolumen. Nie odnotowano QUIC ani jawnego HTTP.

6.9.2 Największe cele (IP/port) wg wolumenu

TABELA 6.20: Największe kierunki ruchu (Voronoi, ta sesja)

IP docelowe	Port	Proto	Flows	MiB
18.245.46.92	443	HTTPS	60	4.37
3.160.212.30	443	HTTPS	17	1.40
216.58.209.3	443	HTTPS	13	1.22
3.98.16.223	443	HTTPS	6	0.33
3.160.212.123	443	HTTPS	6	0.32
35.182.46.15	443	HTTPS	11	0.28
99.79.136.123	443	HTTPS	10	0.18
3.97.54.246	443	HTTPS	1	0.15

Obserwacja: Największy wolumen skupia się na 443/TCP do chmury (adresy AWS/Google) — typowe dla API/CDN aplikacji.

6.9.3 Najczęstsze hosty (liczba przepływów)

TABELA 6.21: Top hosty (Voronoi, liczba przepływów)

Host	Flows
api.voronoiapp.com	221
cdn.voronoiapp.com	182
fonts.gstatic.com	15
firebase logging-pa.googleapis.com	14
firestore.googleapis.com	10
cognito-identity.ca-central-1.amazonaws.com	6
crashlyticsreports-pa.googleapis.com	5
cognito-idp.ca-central-1.amazonaws.com	4
firebase-settings.crashlytics.com	2
firebaseinstallations.googleapis.com	2

Obserwacja: Dominują domeny aplikacji (api.voronoiapp.com, cdn.voronoiapp.com); obecne są usługi Google Firebase/Crashlytics oraz Amazon Cognito (tożsamość).

6.9.4 Obserwacje jakościowe

- **Brak QUIC/HTTP/3** — transport klasyczny TLS/TCP po 443.
- **Brak HTTP w badanym oknie czasowym** — w badanym oknie nie zaobserwowano nieszyfrowanych żądań.
- **Integracje zewnętrzne** — logowanie/konfiguracja przez cognito-identity/cognito-idp; telemetria do crashlytics; logowanie zdarzeń do firebase logging.

6.9.5 Wyniki z Exodus Privacy [5]

Raport dla com.voronoi.organization.app (wersja 1.0.34, Google Play) ujawniła obecność **3 trackerów** oraz **13 uprawnień**.

Trackery:

- **Facebook Share SDK** — integracja społecznościowa, potencjalnie przesyłająca metadane o interakcjach,
- **Google CrashLytics** — monitorowanie stabilności i raportowanie awarii,
- **Google Firebase Analytics** — analiza użycia aplikacji i zachowań użytkowników.

Uprawnienia — przykłady kluczowe:

- **Pamięć masowa:** READ_EXTERNAL_STORAGE, WRITE_EXTERNAL_STORAGE,
- **Reklama/atrybucja:** ACCESS_AD SERVICES_AD_ID, ACCESS_AD SERVICES_ATTRIBUTION,
- **Sieć/system:** INTERNET, ACCESS_NETWORK_STATE, POST_NOTIFICATIONS, READ_GSERVICES,
- **Inne:** RECEIVE (FCM push).

Spostrzeżenia:

- Exodus bazuje na analizie statycznej APK — obecność trackerów oznacza włączenie ich bibliotek, ale nie przesądza o faktycznej aktywności w danej sesji.

- Zidentyfikowane trackery pełnią głównie rolę diagnostyczną i analityczną; zakres uprawnień jest stosunkowo wąski i dotyczy przede wszystkim pamięci, sieci oraz powiadomień.

6.9.6 Subiektywne spostrzeżenia

Obserwowalność treści. Szyfrowanie TLS ogranicza wgląd do metadanych; wolumen wskazuje na standardowe korzystanie z API/CDN (brak jawnego przesyłu danych wrażliwych).

Uwagi dot. bezpieczeństwa. Zachowanie sieciowe (HTTPS, brak plaintext, integracja z Cognito) jest spójne z typową architekturą: tożsamość w AWS Cognito, treści przez CDN/API. Pełna ocena (pinning certyfikatów, obsługa błędów TLS, polityka uprawnień w runtime) wymaga analizy APK/testów dynamicznych.

6.9.7 Podsumowanie

W badanej sesji Voronoi wykazuje:

1. dominację HTTPS (≈ 9.61 MiB) w **465** flow, okno 5.6 min,
2. koncentrację na hostach aplikacji (`api.voronoiaapp.com`, `cdn.voronoiaapp.com`) i usługach Google/AWS (Firebase, Crashlytics, Cognito),
3. brak śladów QUIC i HTTP w badanym oknie czasowym.

6.10 TikTok (wersja 34.8.4)

Okno i wolumen

Okno eksperymentu: **2025-08-24 16:40:55–2025-08-24 16:45:00** (≈ 4.1 min). Zarejestrowano **649** przepływy, wolumen $\approx$ **125.30 MiB** (RX $\approx$ **123.71 MiB**, TX $\approx$ **1.59 MiB**).

Przebieg badania: Scenariusz obejmował: *2 min nasłuchu tła, tworzenie konta, logowanie, wylogowanie, nawigację po głównych stronach, lajkowanie, zmianę personalizacji treści, wyłączenie aplikacji oraz 10 min nasłuchu tła.*

6.10.1 Wolumen i liczność wg protokołu

TABELA 6.22: Podsumowanie wg protokołu (TikTok, ta sesja)

Protokół	Flows	MiB (total)	B wysłane	B odebrane
QUIC	62	2.97	674,032	2,442,153
HTTPS	156	122.24	960,142	127,220,147
TCP	1	0.00	100	88
DNS	238	0.07	17,189	52,366
TLS	190	0.02	12,237	7,919
UDP	2	0.00	2,696	140
Razem	649	125.30	1,666,396	129,722,813

Obserwacja: Wolumenowo dominował HTTPS/TCP.

6.10.2 Największe cele (IP/port) wg wolumenu

TABELA 6.23: Największe kierunki ruchu (TikTok, ta sesja)

IP docelowe	Port	Proto	Flows	MiB
95.101.61.17	443	HTTPS	3	38.58
2.16.168.49	443	HTTPS	7	28.50
213.158.199.80	443	HTTPS	1	21.62
95.101.61.29	443	HTTPS	2	9.48
213.158.199.81	443	HTTPS	9	6.04
23.64.12.178	443	HTTPS	3	3.87
139.177.245.196	443	QUIC	62	2.97
213.158.203.201	443	HTTPS	9	2.74
146.75.118.73	443	HTTPS	5	2.70
18.66.112.33	443	HTTPS	4	1.71

Obserwacja: Największy wolumen kierowany jest do CDN/edge ByteDance i Google (443/UDP i 443/TCP).

6.10.3 Najczęstsze hosty (liczba przepływów)

TABELA 6.24: Top hosty (TikTok, liczba przepływów)

Host	Flows
aggr32-normal.tiktokv.eu	132
libra16-normal-no1a.tiktokv.eu	41
p16-common-sign-useastred.tiktokcdn-eu.com	40
v16m.tiktokcdn-eu.com	32
v19.tiktokcdn-eu.com	20
frontier16-normal-no1a.tiktokv.eu	20
(brak)	19
api16-normal-no1a.tiktokv.eu	16
lf16-pitaya-pkg-sign.tiktokcdn-eu.com	13
p77-sg.tiktokcdn.com	10

Obserwacja: Typowe domeny: *.tiktokv.eu/com, *.tiktokcdn-eu.com, *.byteoversea.com, *.ibytedtos.com (media/CDN, API), mcs-*/mon (telemetria), *.googleapis.com (usługi pomocnicze).

6.10.4 Obserwacje jakościowe

- **Dominacja HTTP/3/QUIC** — strumień wideo po UDP/443; niskie RTT ustanawiania i dobra tolerancja strat.
- **Asymetria RX/TX** — silna przewaga danych odbieranych, zgodna z autoplay/scroll feedu.
- **Telemetria** — połączenia do hostów mcs-*/mon-*; wolumen niewielki względem mediów.

6.10.5 Wyniki z Exodus Privacy [5]

Raport dla com.ss.android.ugc.trill (wersja 34.8.4, Google Play) wskazuje obecność **5 trackerów** — *AppsFlyer* (analitika/atrybucja), *Facebook Login* (identyfikacja), *Facebook Share* (udostępnianie), *Google Firebase Analytics* (analitika) oraz *Pangle* (reklama) — oraz **44 uprawnień**.

Uprawnienia — przykłady kluczowe:

- **Lokalizacja i łączność:** ACCESS_FINE_LOCATION, ACCESS_COARSE_LOCATION, ACCESS_NETWORK_STATE, ACCESS_WIFI_STATE, BLUETOOTH_CONNECT,
- **Multimedia:** RECORD_AUDIO, CAMERA, READ_MEDIA_AUDIO/IMAGES/VIDEO,
- **Powiadomienia/system:** POST_NOTIFICATIONS, FOREGROUND_SERVICE, WAKE_LOCK, SYSTEM_ALERT_WINDOW,
- **Reklama/identyfikacja:** ACCESS_AD SERVICES_AD_ID, AD_ID,
- **Pamięć:** READ/WRITE_EXTERNAL_STORAGE.

6.10.6 Subiektywne spostrzeżenia

Obserwowalność treści. Ze względu na QUIC/TLS treści żądań/odpowiedzi nie są widoczne; analiza dotyczy metadanych (hosty, porty, wolumeny).

Uwagi dot. bezpieczeństwa. Zachowanie sieciowe (HTTP/3 dla mediów, HTTPS/TCP dla API) jest typowe dla aplikacji wideo. Ocena pinningu certyfikatów, DRM i mechanizmów anti-tamper wymaga analizy APK oraz testów dynamicznych.

6.10.7 Podsumowanie

W badanej sesji TikTok wykazuje:

1. dominację QUIC/HTTP/3 w wolumenie (media po UDP/443), przy API po HTTPS/TCP,
2. silną asymetrię RX/TX typową dla streamingu i przewijania feedu,
3. zestaw trackerów i szeroki zakres uprawnień zgodny z raportem Exodusa (5/44).

Rozdział 7

Podsumowanie i wnioski końcowe

7.1 Synteza wyników

Celem pracy była empiryczna charakterystyka ruchu sieciowego wybranych aplikacji Android oraz ocena ryzyk prywatności i bezpieczeństwa na podstawie obserwowalnych metadanych (protokoły, hosty, porty, wolumeny) oraz analizy statycznej (trackery, uprawnienia). W toku badań:

- zarejestrowano i zestandaryzowano sesje pracy aplikacji (Instagram, Facebook, Posnania, mObywatel, USOS, Spotify, Voronoi, TikTok),
- porównano protokoły transportowe (HTTP/1.1, HTTP/2, HTTP/3/QUIC, TLS), koncentrację ruchu na hostach oraz obecność telemetrii,
- skorelowano obserwacje z wynikami z Exodus Privacy (liczba trackerów i kategorie uprawnień).

Najważniejsze tendencje:

1. **Aplikacje multimedialne/społecznościowe** (Instagram, Facebook, TikTok) wykazują wyraźną dominację **HTTP/3/QUIC** i dużą asymetrię RX/TX (konsumpcja treści).
2. **Aplikacje instytucjonalne** (mObywatel, USOS) komunikują się konsekwentnie po **TLS/HTTPS** (brak QUIC, brak HTTP w klarze w zarejestrowanych oknach) i mają ograniczone odwołania do domen trzecich.
3. **Aplikacje komercyjne** (Posnania, Spotify, Voronoi) opierają się głównie na **TLS/HTTPS** z typową telemetrią (np. Sentry, Crashlytics) i zapleczem w chmurze (AWS/Google).

TABELA 7.1: Porównanie badanych aplikacji — zwięzły przegląd sesji i protokołów

Aplikacja	Flows	MiB	Okno	Dominujący	HTTP/3	Uwagi (hosty/telemetria)
Instagram	302	28,04	~12,2 min	QUIC	Tak	graph/gateway.instagram, edge-mqtt (5222); silna asymetria RX/TX
Facebook	305	29,75	~12,3 min	QUIC	Tak	graph/gateway, edge-mqtt; ~98,5% wolumenu po QUIC
Posnania	21	0,08	~2,9 min	HTTPS	Nie	yourb.app; Sentry (ingest.sentry.io); Stripe
mObywatel	13	0,21	~2,9 min	HTTPS/TLS	Nie	api.mobywatel.gov.pl; sentry.puch.coi.gov.pl; jednorazowy HTTP/80 do Google
USOS	200	2,88	~4,3 min	HTTPS	Nie	usosapps.put.poznan.pl, ppulse.put.poznan.pl; Crashlytics/Firebase
Spotify	72	12,42	~2,2 min	HTTPS	Nie	spclient.wg.spotify.com; mDNS/SSDP (integracje urządzeń)
Voronoi	465	9,61	~5,6 min	HTTPS	Nie	adresy AWS/Google (API/CDN); TLS/3000 śladowy
TikTok	649	125,30	~4,1 min	HTTPS (+ QUIC)	Tak	*tiktokv*.eu / tiktokcdn-eu.com; QUIC dla części strumieni; silna asymetria RX/TX

Dane pochodzą z pojedynczych sesji opisanych w rozdziale eksperymentów.

7.2 Wkład rozdziału „Pierwsze spostrzeżenia...”

Przed badaniem właściwym wykonano serię prób i inwentaryzację, które bezpośrednio ukształtowały metodykę:

Inwentaryzacja aplikacji po resecie. Na dwóch urządzeniach po przywróceniu ustawień fabrycznych zidentyfikowano **53** domyślne aplikacje. Dla **44** (83,0%) można było ręcznie *zablokować* transfer w tle, dla **9** (17,0%) — nie. Domyślnie **100%** aplikacji miało *aktywny* transfer w tle. Wniosek praktyczny: *uprzednie* ograniczenie ruchu w tle (gdzie to możliwe) zmniejsza szum pomiarowy i poprawia rozdzielczość obserwacji scenariuszowych.

TABELA 7.2: Ustawienia transferu danych w tle dla aplikacji domyślnych (po resecie)

Kategoria	Liczba	Udział
Łączna liczba aplikacji	53	100%
Można zablokować dane w tle	44	83,0%
Nie można zablokować	9	17,0%
Domyślnie aktywny transfer w tle	53	100%
Domyślnie zablokowany transfer	0	0%

Przechwytywanie radiowe (Wireshark/monitor mode) — niepowodzenie. Sniffing ramek 802.11 z próbą dekrypcji ruchu aplikacji okazał się nieefektywny (złożone warunki radiowe, rotacja kluczy, brak pełnego materiału kryptograficznego). **Wniosek:** metoda ma wysoką cenę

operacyjną i niską wartość poznawczą dla aplikacji mobilnych w warunkach produkcyjnych; lepiej *kierować ruch przez własny węzeł* i rejestrować go na końcu.

MITM przez proxy (mitmproxy) — ograniczenia. Podejście z własnym AP i proxy ułatwia klasyfikację hostów i kontrolę DNS, ale w praktyce na przeszkodzie stają **pinning certyfikatów** i **HTTP/3/QUIC**, które omijają klasyczny „TLS-over-TCP” MITM.

MITM + instrumentacja (Frida) — próba nieudana. Próba dynamicznego obejścia niestandardowych walidacji certyfikatów i pinningu zakończyła się niepowodzeniem (niestandardowe ścieżki weryfikacji, komponenty spoza prostych hooków). **Wniosek:** skuteczny MITM nowoczesnych aplikacji wymaga *silnie customizowanej* instrumentacji lub środowisk buildowych — co wykracza poza zakres niniejszej pracy.

Rejestracja na urządzeniu (PCAP/PCAPNG) — sukces. Rejestracja bezpośrednio na telefonie (np. PCAPdroid) okazała się **najbardziej efektywna**, dając stabilny wgląd w metadane (protokoły/hosty/wolumeny) przy zachowaniu realnych warunków użytkowania, mimo braku semantyki HTTP przy QUIC/pinningu.

7.3 Wnioski merytoryczne

1. **HTTP/3/QUIC jako standard mediów:** aplikacje z intensywną warstwą treści (Instagram, Facebook, TikTok) niemal całkowicie przeniosły strumienie na QUIC, co ogranicza możliwości klasycznego MITM i przesunęło analizę w stronę metadanych.
2. **Transport TLS/HTTPS jako norma bezpieczeństwa:** w sesjach mObywatel i USOS nie odnotowano jawnego HTTP ani QUIC; ruch koncentrował się na hostach instytucjonalnych z niewielką telemetrią pomocniczą.
3. **Telemetria w aplikacjach komercyjnych:** Posnania (Sentry, Stripe) i Spotify (ograniczona telemetria Google/Crashlytics) pokazują typowy profil integracji; Voronoi korzysta z chmury (AWS/Google) bez jawnego QUIC.
4. **Ryzyka funkcjonalne niezależne od transportu:** w Posnaniu zidentyfikowano możliwość *dodania dowolnej tablicy rejestracyjnej* i wglądu w zdarzenia parkingowe — to obserwacja wymagająca dalszych badań on-device (nie przesądzana przez sam ruch sieciowy), ale istotna z perspektywy prywatności.

7.4 Ograniczenia badania

- wartości liczbowe dotyczą *konkretnych wersji* aplikacji i *pojedynczych okien czasowych*; zmiany wersji i regionu mogą wpływać na protokoły i hosty,
- QUIC i pinning ograniczają wgląd w semantykę HTTP — analiza dotyczy głównie metadanych,
- wyniki Exodus Privacy są statyczne (sygnatury obecności trackerów), a nie dowodem ich aktywności w danej sesji.

7.5 Rekomendacje i kierunki dalszych prac

1. **Metodyka:** w badaniach terenowych preferować *rejestrację on-device* + pasywną klasyfikację, a MITM stosować selektywnie (tam, gdzie brak QUIC i pinning pozwala na wgląd w HTTP).
2. **Automatyzacja:** rozbudować pipeline o rozpoznawanie kategorii hostów (API/CDN/telemetria) i detekcję anomalii (np. nietypowe porty, nagły wzrost domen trzecich).
3. **Prywatność:** dla aplikacji instytucjonalnych utrzymywać monitoring odwołań do domen trzecich i okresowo weryfikować politykę uprawnień.
4. **Weryfikacja hipotez funkcyjnych:** przeprowadzić testy on-device (np. inspekcja bazy lokalnej, pinning, reverse engineering APK) dla przypadków takich jak rejestracje pojazdów w Poznaniu.

7.6 Konkluzja

Zastosowana metodyka pozwoliła na **spójne i powtarzalne** porównanie zachowań sieciowych różnych klas aplikacji. W praktyce użytkowej **transport jest w przeważającej mierze szyfrowany** (TLS/HTTPS lub QUIC), zaś realne ryzyka prywatności ujawniają się częściej w *semantyce funkcji* i *modelu danych* niż w „nagiej” warstwie protokołów. Badania potwierdzają kierunek: *metadane są wystarczające do profilowania zachowań aplikacji*, ale *ocena bezpieczeństwa* wymaga łączenia pasywnej obserwacji z testami dynamicznymi i analizą on-device.

Bibliografia

- [1] Android Open Source Project. „Application Sandbox.” (2025), adr.: <https://source.android.com/docs/security/app-sandbox> (term. wiz. 17.08.2025).
- [2] A. Razaghpanah, R. Nithyanand, N. Vallina-Rodriguez i in., „Apps, Trackers, Privacy, and Regulators: A Global Study of the Mobile Tracking Ecosystem,” w *Network and Distributed System Security Symposium (NDSS)*, 2018. adr.: https://www.ndss-symposium.org/wp-content/uploads/2018/02/ndss2018_05B-3_Razaghpanah_paper.pdf.
- [3] Android Open Source Project. „Architecture overview.” (2025), adr.: <https://source.android.com/docs/core/architecture> (term. wiz. 17.08.2025).
- [4] „Cisco Annual Internet Report (2018–2023) White Paper,” Cisco, 2020, Prognozy do 2023 r.; m.in. penetracja subskrypcji mobilnych (71% global population with mobile subscription by 2023). adr.: <https://www.cisco.com/c/en/us/solutions/collateral/executive-perspectives/annual-internet-report/white-paper-c11-741490.html> (term. wiz. 17.08.2025).
- [5] *Exodus Privacy: Analysis of trackers and permissions in Android apps*, <https://exodus-privacy.eu.org/>, Accessed 2025-08-17, 2025.
- [6] *Frida: A world-class dynamic instrumentation toolkit*, <https://frida.re/docs/home/>, Accessed 2025-08-17, 2025.
- [7] StatCounter, *Global Platform Market Share: Desktop vs Mobile vs Tablet*, Udział globalnych page views według platformy; potwierdza >50% dla urządzeń mobilnych (sprawdź dokładny zakres dat), 2025. adr.: <https://gs.statcounter.com/platform-market-share/desktop-mobile-tablet> (term. wiz. 17.08.2025).
- [8] Google. „How We Kept the Google Play and Android App Ecosystem Safe in 2024.” Android app ecosystem security overview. (sty. 2025), adr.: <https://security.googleblog.com/2025/01/how-we-kept-google-play-android-app-ecosystem-safe-2024.html> (term. wiz. 17.08.2025).
- [9] *HTTP/2*, RFC 9113, 2022. adr.: <https://www.rfc-editor.org/rfc/rfc9113>.
- [10] M. Bishop, *HTTP/3*, RFC 9114, 2022. adr.: <https://www.rfc-editor.org/rfc/rfc9114>.
- [11] OWASP Mobile Security Testing Guide. „MASTG-TECH-0120: Intercepting HTTP Traffic Using an Interception Proxy.” (2025), adr.: <https://mas.owasp.org/MASTG/techniques/generic/MASTG-TECH-0120/> (term. wiz. 17.08.2025).
- [12] OWASP Mobile Security Testing Guide. „MASTG-TEST-0244: Missing Certificate Pinning in Network APIs.” (2025), adr.: <https://mas.owasp.org/MASTG/tests/android/MASVS-NETWORK/MASTG-TEST-0244/> (term. wiz. 17.08.2025).
- [13] *mitmproxy: Interactive TLS-capable intercepting HTTP proxy*, <https://mitmproxy.org/>, Accessed 2025-08-17, 2025.

- [14] mitmproxy project. „mitmproxy — About Certificates.” (2025), adr.: <https://docs.mitmproxy.org/stable/concepts/certificates/> (term. wiz. 17.08.2025).
- [15] *Mobile Security Framework (MobSF) Documentation*, <https://github.com/MobSF/docs>, Accessed 2025-08-17, 2025.
- [16] Android Developers. „Permissions on Android — Overview.” (2025), adr.: <https://developer.android.com/guide/topics/permissions/overview> (term. wiz. 17.08.2025).
- [17] Android Developers. „Platform architecture.” (2024), adr.: <https://developer.android.com/guide/platform> (term. wiz. 17.08.2025).
- [18] J. M. del Álamo, J. López-Fernández i in., „Privacy Assessment in Android Apps: A Systematic Mapping Study,” *Electronics*, t. 10, nr. 16, s. 1999, 2021. adr.: <https://www.mdpi.com/2079-9292/10/16/1999>.
- [19] Google. „Privacy Sandbox on Android — Overview.” (2025), adr.: <https://privacysandbox.google.com/overview/android> (term. wiz. 17.08.2025).
- [20] *Profile battery usage with Batterystats and Battery Historian*, <https://developer.android.com/topic/performance/power/setup-battery-historian>, Accessed 2025-08-17, 2024.
- [21] *Profile your app performance (Android Studio Profiler)*, <https://developer.android.com/studio/profile>, Accessed 2025-08-17, 2024.
- [22] *QUIC: A UDP-Based Multiplexed and Secure Transport*, RFC 9000, 2021. adr.: <https://www.rfc-editor.org/rfc/rfc9000>.
- [23] Android Developers. „Request runtime permissions.” (2025), adr.: <https://developer.android.com/training/permissions/requesting> (term. wiz. 17.08.2025).
- [24] V. F. Taylor, R. Spolaor, M. Conti i I. Martinovic, „Robust Smartphone App Identification via Encrypted Network Traffic Analysis,” *arXiv preprint arXiv:1704.06099*, 2017. adr.: <https://arxiv.org/pdf/1704.06099>.
- [25] Android Developers. „SdkSandboxManager API reference.” (2025), adr.: <https://developer.android.com/design-for-safety/privacy-sandbox/reference/sdksandbox/SdkSandboxManager> (term. wiz. 17.08.2025).
- [26] *tcpdump/libpcap*, <https://www.tcpdump.org/>, Accessed 2025-08-17, 2025.
- [27] M. Conti, Q. Li, A. Maragno i R. Spolaor, „The Dark Side(-Channel) of Mobile Devices: A Survey on Network Traffic Analysis,” *arXiv preprint arXiv:1708.03766*, 2017. adr.: <https://arxiv.org/abs/1708.03766>.
- [28] E. Rescorla, *The Transport Layer Security (TLS) Protocol Version 1.3*, RFC 8446, 2018. adr.: <https://www.rfc-editor.org/rfc/rfc8446>.
- [29] A. Pathak, Y. C. Hu i M. Zhang, „Where is the Energy Spent in Android Apps? Fine-Grained Energy Accounting on Smartphones with EProf,” w *Proceedings of the 7th ACM European Conference on Computer Systems (EuroSys '12)*, ACM, 2012.
- [30] S. Fahl, M. Harbach, T. Muders, L. Baumgärtner, B. Freisleben i M. Smith, „Why Eve and Mallory Love Android: An Analysis of Android SSL (In)Security,” w *Proceedings of the 2012 ACM Conference on Computer and Communications Security (CCS '12)*, ACM, 2012. adr.: <https://saschafahl.de/static/paper/androidssl2012.pdf>.
- [31] Wireshark Foundation. „Wireshark User’s Guide.” (2025), adr.: <https://www.wireshark.org/docs/> (term. wiz. 17.08.2025).

Spis tabel

4.1	Mapa elementów badania do narzędzi i metryk	21
5.1	Podsumowanie ustawień korzystania z danych sieciowych w tle (po resecie)	23
5.2	Lista aplikacji, dla których nie można zablokować danych sieciowych w tle (po resecie)	23
6.1	Podsumowanie wg protokołu (Instagram, ta sesja)	28
6.2	Największe kierunki ruchu (Instagram, ta sesja)	28
6.3	Top hosty (Instagram, liczba przepływów)	29
6.4	Podsumowanie wg protokołu (Facebook, ta sesja)	31
6.5	Największe kierunki ruchu (Facebook, ta sesja)	31
6.6	Top hosty (Facebook, liczba przepływów)	32
6.7	Podsumowanie wg protokołu (Posnania, ta sesja)	34
6.8	Największe kierunki ruchu (Posnania, ta sesja)	34
6.9	Top hosty (Posnania, liczba przepływów)	34
6.10	Podsumowanie wg protokołu (mObywatel, ta sesja)	36
6.11	Największe kierunki ruchu (mObywatel, ta sesja)	36
6.12	Top hosty (mObywatel, liczba przepływów)	36
6.13	Podsumowanie wg protokołu (USOS, ta sesja)	39
6.14	Największe kierunki ruchu (USOS, ta sesja)	39
6.15	Top hosty (USOS, liczba przepływów)	40
6.16	Podsumowanie wg protokołu (Spotify, ta sesja)	42
6.17	Największe kierunki ruchu (Spotify, ta sesja)	42
6.18	Top hosty (Spotify, liczba przepływów)	43
6.19	Podsumowanie wg protokołu (Voronoi, ta sesja)	45
6.20	Największe kierunki ruchu (Voronoi, ta sesja)	45
6.21	Top hosty (Voronoi, liczba przepływów)	46
6.22	Podsumowanie wg protokołu (TikTok, ta sesja)	48
6.23	Największe kierunki ruchu (TikTok, ta sesja)	48
6.24	Top hosty (TikTok, liczba przepływów)	49
7.1	Porównanie badanych aplikacji — zwięzły przegląd sesji i protokołów	52
7.2	Ustawienia transferu danych w tle dla aplikacji domyślnych (po resecie)	52

(2025)

Instytut Informatyki, Wydział Informatyki i Telekomunikacji
Politechnika Poznańska

Skład przy użyciu systemu L^AT_EX.